Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

Identity Security Framework for Healthcare

Student Name: SUDHEESH KS

Industry Mentor: Mr. Shashank

Guided by: Ms. Divya N Shetty

Table of Contents

1 BACKGROUND

The healthcare sector requires secure and efficient systems to manage patient records, employee data, and hospital operations. Traditional systems often lack strong security mechanisms, making them vulnerable to unauthorized access and cyber threats.

To address these issues, modern systems are designed with integrated security frameworks that ensure safe handling of sensitive medical data. This project introduces a secure, web-based healthcare system that combines identity management, authentication, and advanced security features such as OTP verification, role-based access control, and anomaly detection.

The system is built using a full-stack architecture with React for the frontend, Node.js and Express.js for the backend, and PostgreSQL for data storage, along with a Python-based ML service for detecting suspicious activities. This ensures both efficient healthcare management and strong protection of user identities.

1.1 Aim

The aim of this project is to design and develop a secure healthcare system that efficiently manages users, employees, and patient-related operations while ensuring strong identity protection and data security. The system focuses on providing secure authentication, role-based access control, and real-time monitoring of user activities using machine learning-based anomaly detection. It aims to improve the safety, reliability, and efficiency of healthcare data management through a scalable web-based architecture. Additionally, the project aims to reduce unauthorized access and cyber threats by implementing multi-layer security mechanisms such as JWT authentication, session management, and OTP verification. It also ensures smooth communication between different system modules using RESTful APIs.

1.2 Technologies

The system is developed using modern web technologies and security tools to ensure scalability, performance, and secure operation. The technology stack consists of frontend, backend, database, machine learning services, authentication mechanisms, and supporting security tools.

- Frontend

React.js is used to build a dynamic and responsive user interface for the system by providing interactive dashboards for employees, patients, and administrators, enabling smooth navigation across modules. It follows a component-based architecture in which the user interface is divided into reusable components, with each component managing its own logic and communicating with other components through props and state. This architecture improves modularity, promotes code reusability, and simplifies development and maintenance. React also supports efficient state management using hooks, which helps manage application data and user interactions effectively. The use of the Virtual DOM improves rendering performance by updating only necessary parts of the interface, enhancing responsiveness and user experience. In addition, React offers several advantages such as fast rendering, scalable structure, easy maintenance, and strong community support. It also integrates effectively with REST APIs, making it suitable for full-stack web applications. Overall, React enhances usability, supports efficient frontend development, and provides a modern and flexible user experience.

- Backend

Node.js with Express.js is used to implement server-side logic and REST APIs. It handles authentication, authorization, request processing, and core business operations. Node.js uses an event-driven, non-blocking architecture, allowing the system to handle multiple requests efficiently. Express.js simplifies routing, middleware integration, and API development. It also supports implementation of security mechanisms such as JWT validation and role-based access control. Together, they provide a fast, scalable, and efficient backend.

- Node.js Runtime Features

Node.js provides asynchronous processing, event-driven architecture, and non-blocking I/O operations, which improve system performance when handling multiple concurrent users and requests efficiently. Its architecture allows the server to process tasks without waiting for one operation to complete before starting another, improving responsiveness and resource utilization. Node.js also supports lightweight execution, making it suitable for fast and efficient application development. Its scalability enables the system to handle increasing workloads and support distributed environments effectively. Package management through npm provides access to a wide range of libraries and tools,

simplifying development and extending functionality. In addition, Node.js supports real-time applications such as live notifications and instant data updates, making it suitable for interactive systems. These capabilities make Node.js a strong choice for building scalable, high-performance, and distributed web applications.

- Express Middleware Use

  Express middleware is used to process requests before they reach the main application logic, allowing important functions to be handled during the request-response cycle. Middleware performs tasks such as authentication checks, input validation, logging, rate limiting, and error handling to improve security and ensure reliable request processing. It helps verify user access, filter invalid or malicious inputs, and track application activities for monitoring and debugging. Security-focused middleware such as Helmet adds protective HTTP security headers, while express-rate-limit controls excessive requests to reduce abuse and denial-of-service risks. Session middleware supports secure session management by maintaining user state and controlling authenticated access across requests. By separating these functions into reusable layers, middleware improves modularity, simplifies maintenance, and strengthens the overall security and performance of the application.

Database

PostgreSQL is used as the relational database management system for storing and managing structured data within the system, including users, employees, patients, lab tests, prescriptions, ward admissions, and system logs. It supports strong relationships between tables and ensures data integrity through constraints, foreign keys, and relational modelling. PostgreSQL enables secure and reliable storage of sensitive healthcare information while supporting consistency and durability of data. It also provides advanced query processing, efficient data retrieval, and transaction management to support system operations. With features such as ACID compliance, indexing, foreign key relationships, and support for complex queries, PostgreSQL ensures reliable and efficient database performance. Additional features including data security, scalability, backup support, and high-performance structured data processing make it suitable for enterprise-level applications.

Overall, PostgreSQL provides a robust, secure, and scalable database foundation for managing critical application data.

- Machine Learning Service – Python Flask

A Python Flask-based microservice is used for anomaly detection. It analyzes user behavior and request patterns to detect suspicious activities. Machine learning models are used to identify threats such as bots, SQL injection attempts, and abnormal login behavior. The service communicates with the backend through REST APIs. Flask makes the service lightweight and easy to integrate. This enhances the overall security of the system.

- ❖ Python Flask ML Service

Flask provides a lightweight framework for exposing machine learning models as APIs and enables communication between the ML service and the backend. It processes request data, evaluates security risks, and returns anomaly detection results. Flask supports simple API integration and efficient handling of prediction requests. Its modular structure makes model updates and future improvements easier. These features make Flask suitable for scalable and flexible machine learning service integration.

- **Authentication – JWT and Session**

The system uses a hybrid authentication mechanism for secure access control. JWT is used for employees to provide stateless authentication and secure API access. Session-based authentication is used for administrators and patients for server-side security control. Tokens and sessions are validated for each request to prevent unauthorized access. This approach protects against session hijacking and unauthorized use. It ensures secure identity management across the system.

- **JWT Features**

JWT provides signed and tamper-resistant tokens used for secure authentication, with tokens containing user identity and access-related information. Key features include stateless authentication, token expiry, secure API access, and support for distributed systems. JWT reduces the need for repeated server-side session storage, improving scalability and performance. It helps verify user requests securely before granting access to protected resources. Token-based authentication also supports secure communication between services in distributed environments. Additionally, JWT can

be integrated with authorization controls to manage role-based access and strengthen overall application security.

➢ Security and Supporting Tools

Security is strengthened using multiple supporting tools integrated into the system. bcrypt is used to securely hash passwords before storing them in the database. OTP verification is used as a second authentication layer during sensitive operations. Helmet protects the system by setting secure HTTP headers. Nodemailer is used to send emails such as OTPs, alerts, and notifications. Together, these tools enhance overall system protection.

➢ Bcrypt

bcrypt uses salted hashing to protect passwords from reverse engineering and prevents storage of plain-text passwords, strengthening credential security. It adds random salts to each password before hashing, making attacks such as rainbow table attacks more difficult. bcrypt also supports configurable hashing rounds, allowing stronger protection by increasing computational complexity. This improves resistance against brute-force attacks and enhances secure user authentication.

➢ Helmet

Helmet secures the application by configuring HTTP security headers and helps prevent attacks such as XSS, clickjacking, and MIME sniffing. It adds multiple protective headers that reduce common web vulnerabilities and strengthen browser-side security. Helmet also supports secure content policies and improves protection against unauthorized script execution. These features enhance the overall security posture of the web application.

➢ Nodemailer

Nodemailer provides email functionality for OTP delivery, password reset links, and account notifications while supporting secure communication between the system and employees. It enables automated email-based verification and employee communication within the application. Nodemailer supports SMTP integration, secure email transmission, and customizable notification handling. These features improve account security, employee verification, and system communication reliability.

.

1.3 Hardware Architecture

The system follows a distributed hardware architecture to ensure reliability, performance, and scalability. It is designed to run across multiple machines or servers, separating frontend, backend, database, and ML services. The client side consists of user devices such as desktops, laptops, or mobile devices, which access the system through a web browser. These devices interact with the frontend application and send requests over the internet. The application server hosts the backend built using Node.js and Express.js. It processes incoming requests, handles authentication and business logic, and communicates with other components of the system. The database server runs PostgreSQL, which stores all structured data including employee information, patient records, and system logs, ensuring secure and efficient data storage and retrieval. The ML service server runs the Python Flask application for anomaly detection, analyzing request data and identifying suspicious activities to enhance system security. All components communicate over a network using HTTP/HTTPS protocols, ensuring secure and efficient data exchange across the system. Firewalls and access control mechanisms protect communication channels and restrict unauthorized access between hardware components. Load balancing can be used to distribute requests across services and improve system performance under high traffic conditions. Backup and recovery mechanisms support data protection and help maintain availability during failures. Logging and monitoring servers assist in tracking system activity, performance, and potential security events. Network segmentation can isolate critical services such as the database and ML servers to improve protection against attacks. Preventive controls are incorporated to reduce security risks and strengthen infrastructure protection. Regular patching and software updates help prevent vulnerabilities from being exploited. Intrusion detection mechanisms support early identification of suspicious behavior, while secure authentication controls protect access to system resources. Redundant server configurations can improve availability by reducing single points of failure. Resource monitoring tools can help manage hardware performance and support capacity planning. Secure server configuration practices further strengthen protection against infrastructure-level threats. This distributed architecture improves fault tolerance, supports scalability, resilience, and enables efficient coordination between all system components.

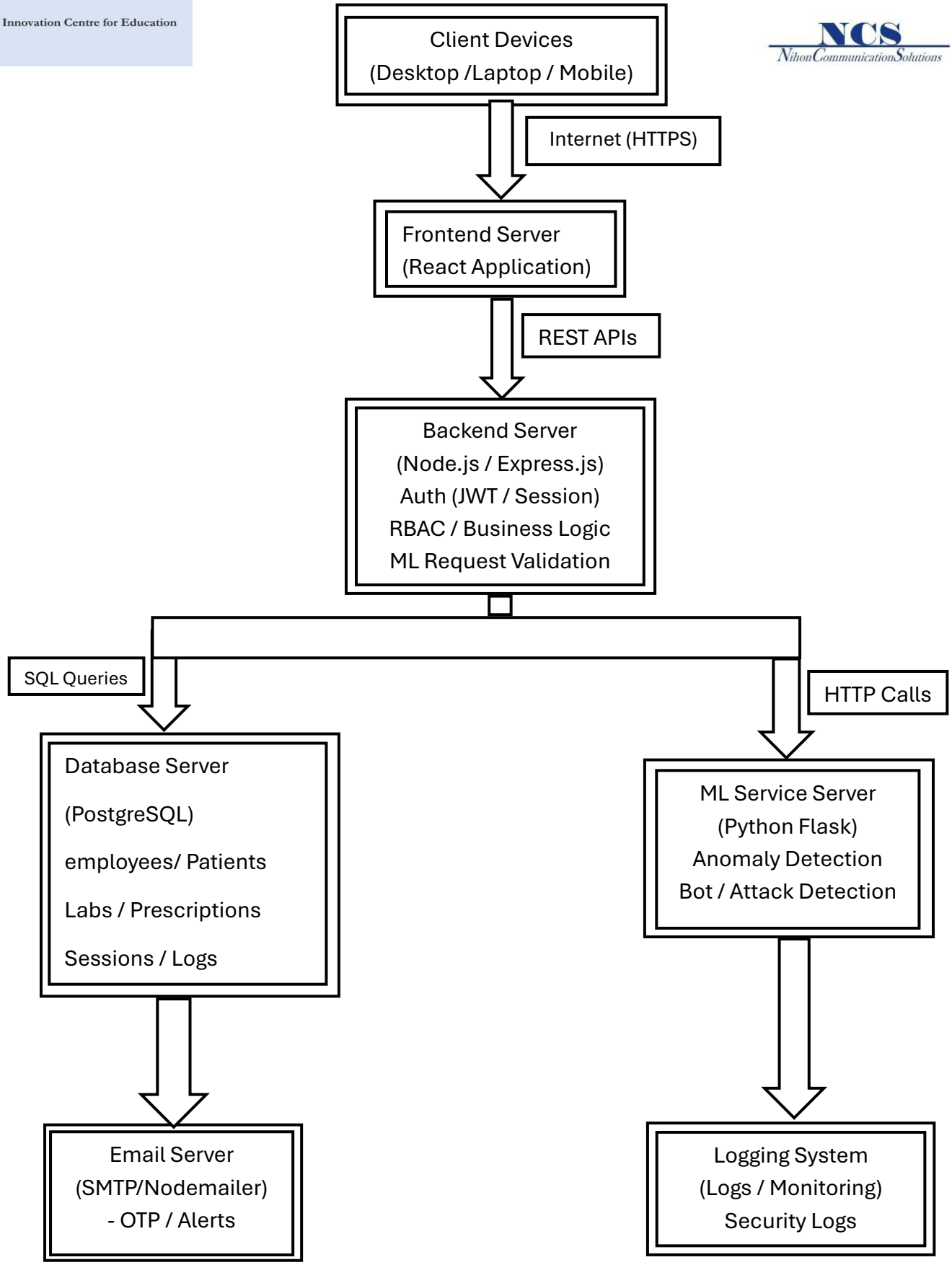


Figure 1.3: Hardware Architecture Diagram

1.3.1 Server Roles

The system architecture is designed with multiple servers, each performing a specific role to ensure efficient operation and clear separation of responsibilities. The Frontend Server hosts the web interface and delivers the client-side application to users, enabling interaction with the system. The Application Server manages core functionalities such as APIs, authentication, business logic, and coordination between services. The Database Server is responsible for storing, managing, and retrieving structured application data securely and efficiently. The ML Server handles anomaly detection by processing data and returning threat analysis results to support security monitoring. Additionally, the Email Server facilitates OTP delivery and sends system notifications for communication and verification purposes. This separation of responsibilities enhances maintainability, supports better load distribution, improves scalability, and increases overall system efficiency.

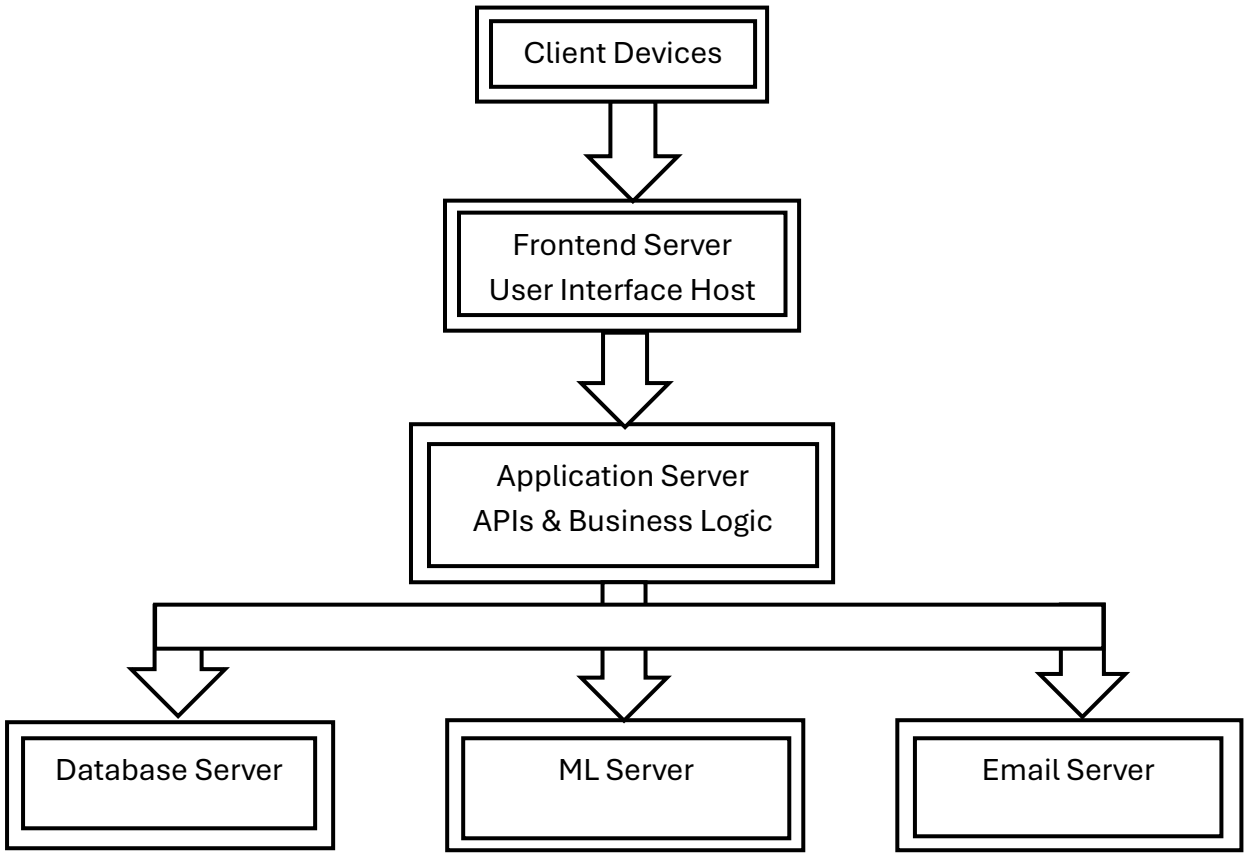


Figure 1.3.1: Server Roles Diagram

1.3.2 Network Communication Flow

The network communication flow defines how system components interact securely and efficiently within the architecture. Client devices communicate with the frontend server using HTTPS, ensuring encrypted transmission of requests and responses. The frontend handles user interactions and communicates with the backend through REST API calls for authentication, data processing, and service requests. The backend applies business logic and interacts with the PostgreSQL database using SQL queries to manage structured data. It also communicates with the machine learning service using HTTP/REST requests for anomaly detection and threat analysis. Email communication, including OTP delivery and notifications, is handled through SMTP protocols. Security controls are integrated into the communication flow through firewalls, access control mechanisms, encryption protocols, and secure authentication methods to protect data confidentiality and restrict unauthorized access. Secure API gateways and request validation help filter requests and prevent malformed data from affecting system operations. Session management controls maintain secure interactions between employees and the application, while intrusion detection mechanisms help identify suspicious traffic patterns. Logging and monitoring tools support auditing, troubleshooting, and security event tracking. Performance and reliability are improved through load balancing, controlled request routing, backup communication paths, and network segmentation. These mechanisms support efficient traffic distribution, fault tolerance, and protection of critical components. Performance monitoring tools track response times and bandwidth usage for optimization, while protocol standardization ensures consistent communication across modules. Redundant communication channels can improve availability during network disruptions. Traffic filtering mechanisms can help block malicious requests before reaching core services. Secure protocol enforcement further strengthens protection for data exchanged across interconnected components. This structured communication flow supports secure data exchange, reliable coordination, scalability, resilience, and efficient overall system performance.

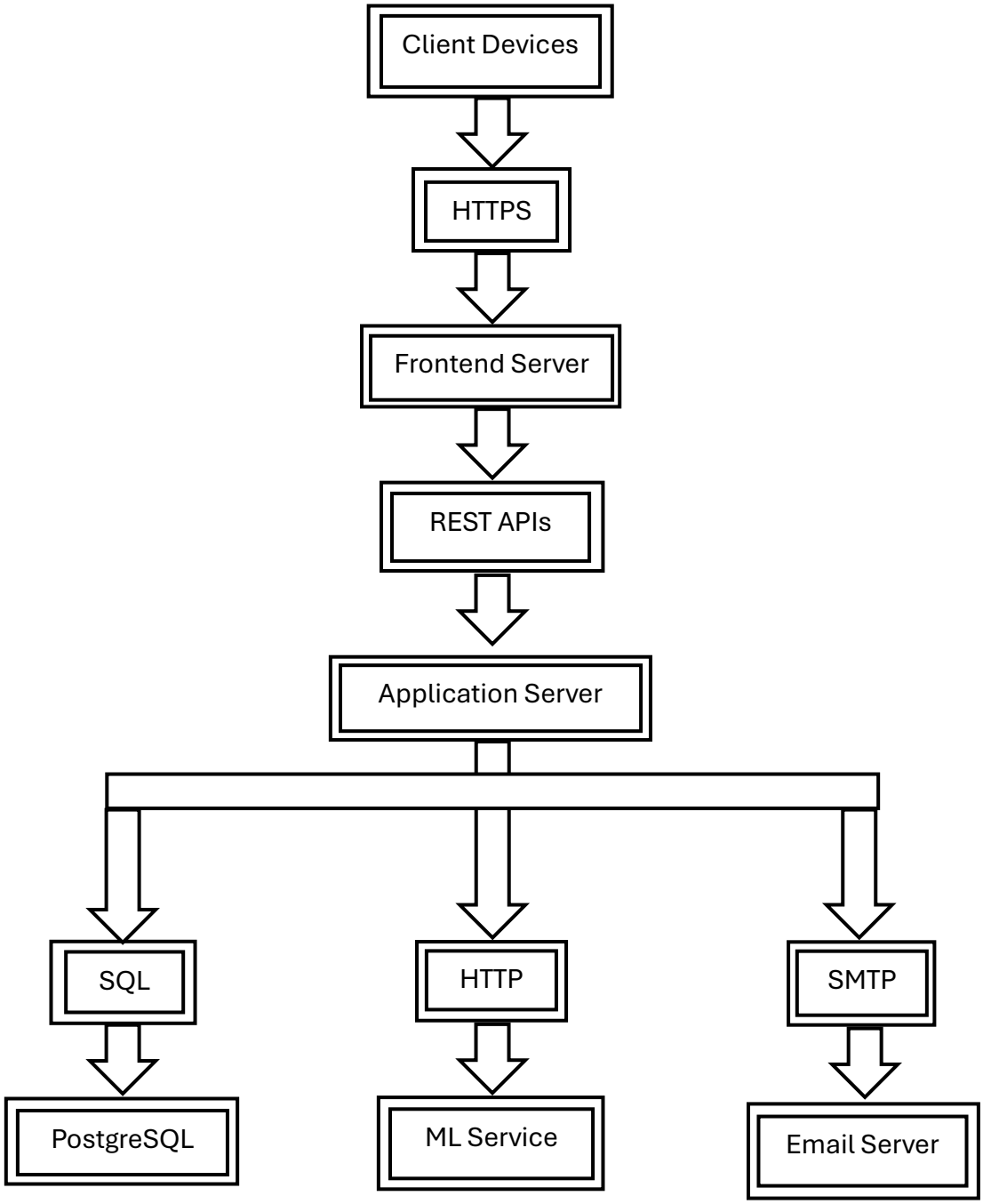


Figure 1.3.2: Network Communication Flow Diagram

1.3.3 Security at Hardware Level

Security is incorporated at the hardware and infrastructure level to protect system components from unauthorized access, attacks, and operational risks. Secure communication is enforced using HTTPS encryption to protect data transmitted between users and system services. Application servers implement authentication controls and request validation to ensure only legitimate users and approved requests are processed. Database servers restrict unauthorized access through access control mechanisms and protect stored data from exposure or tampering. Machine learning service servers help identify suspicious requests, abnormal behaviour, and potential attacks by supporting threat detection processes. Logging and monitoring mechanisms track security events, record system activities, and support auditing and incident response. Firewalls provide an additional layer of protection by filtering malicious traffic before it reaches internal components. Intrusion detection mechanisms help monitor network behaviour and identify potential threats in real time. Backup and recovery controls support system resilience in the event of failures or attacks. These infrastructure-level security controls strengthen overall system protection and improve defence against hardware and network-based threats.

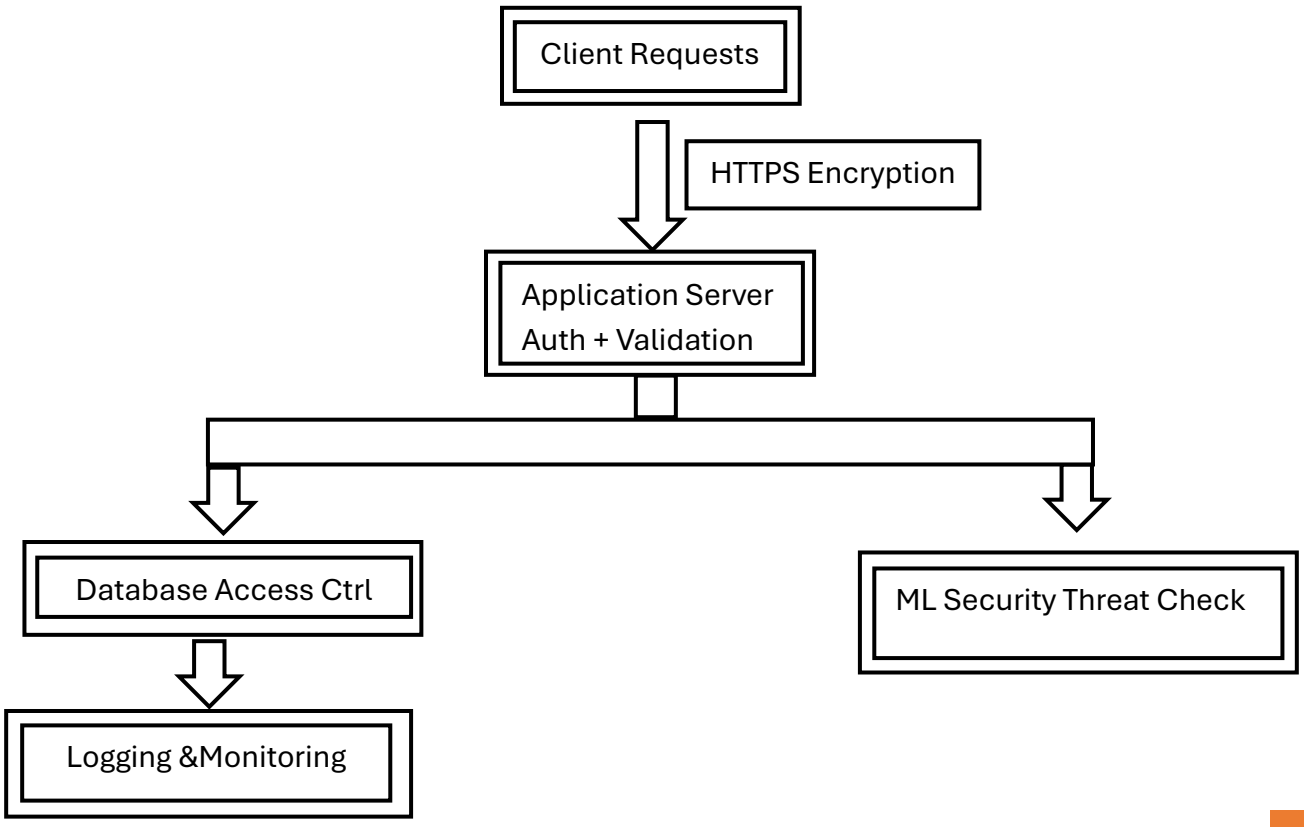


Figure 1.3.3: Security at Hardware Level Diagram

1.4 Software Architecture

The system uses a three-tier software architecture to ensure modularity, scalability, and security by separating the system into presentation, application, and data layers. The Presentation Layer, built with React, provides the user interface, handles user interactions, and communicates with the backend through REST APIs. The Application Layer, developed using Node.js and Express.js, manages business logic, authentication, authorization, API routing, and security functions, including JWT, session management, OTP verification, and ML-based anomaly detection. The Data Layer uses PostgreSQL to securely store and manage structured data while ensuring integrity and efficient retrieval. Supporting services such as a Flask-based ML service and Email Service enhance threat detection, notifications, and overall system communication. This layered structure improves maintainability by separating responsibilities across components. It also supports efficient integration of security controls and simplifies future system expansion. The architecture enables reliable performance and coordinated communication between all modules.

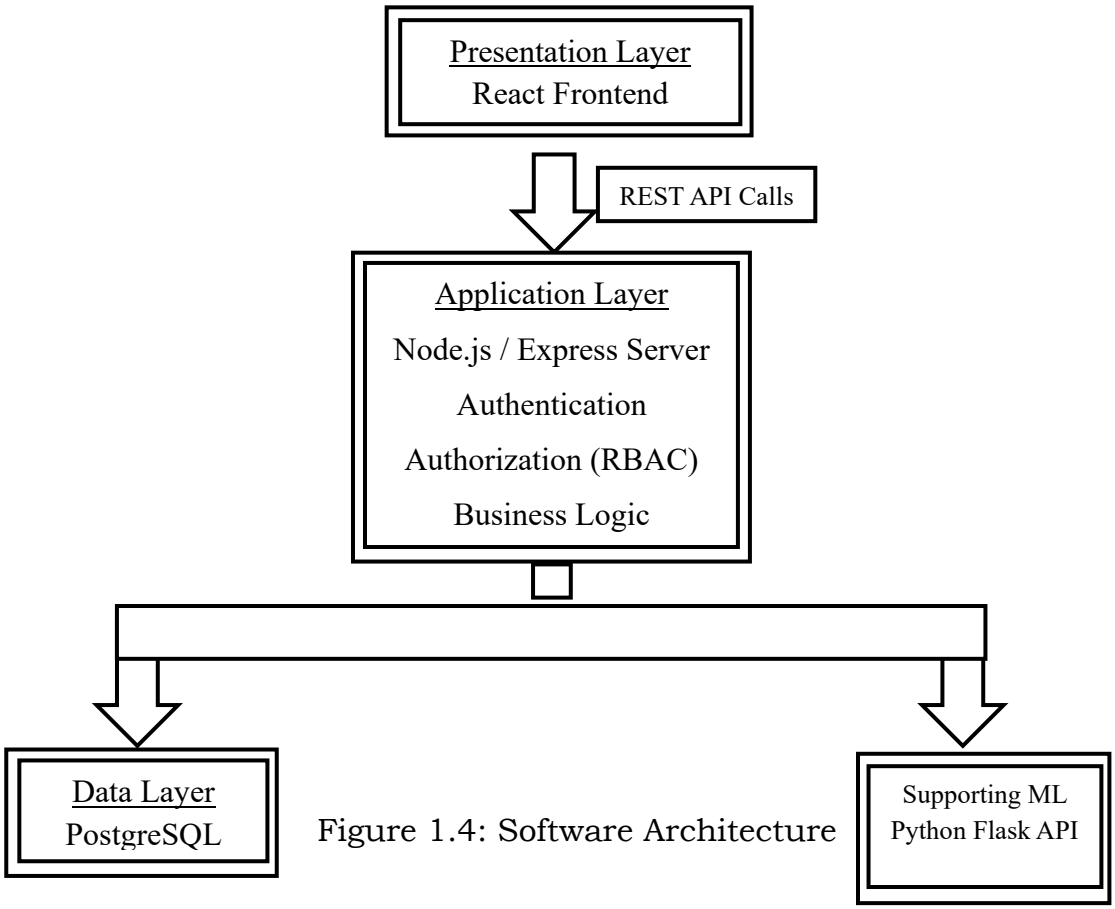


Figure 1.4: Software Architecture

2 System

The system is a secure, web-based platform designed to manage healthcare operations and protect user identities. It supports key functionalities such as employee management, patient management, lab tests, prescriptions, and ward admissions. The system ensures efficient handling of data while maintaining high security standards. It is built using a full-stack architecture with React for the frontend, Node.js and Express.js for the backend, and PostgreSQL for data storage. The system also integrates a Python Flask-based ML service for anomaly detection to identify suspicious activities. This combination ensures scalability, performance, and reliability.The system implements a hybrid authentication mechanism using JWT for employees and session-based authentication for administrators and patients. Additional security features include OTP verification, role-based access control (RBAC), and protection against common cyber threats. Overall, the system provides a secure, efficient, and scalable solution for healthcare management.

2.1 Requirements

The system requirements define the necessary functionalities and conditions needed for the successful operation of the application. These are divided into functional and non-functional requirements.

2.1.1 Functional Requirements

The system must provide secure authentication for employees, administrators, and p atients using JWT and session-based mechanisms. It should support core functionalities such as employee management, patient registration, lab tests, prescriptions, and ward admissions. The system must implement role-based access control (RBAC) to restrict access based on user roles. It should include OTP verification for password reset and sensitive operations. The system must detect suspicious activities using ML-based anomaly detection. All system activities such as login attempts and errors should be logged for monitoring.

2.1.2 User Requirements

Users should be able to easily access the system through a web browser with a simple and user-friendly interface. Employees should have access to role-specific dashboards and functionalities. Patients should be able to register, log in, and view their medical records. Administrators should be able to manage users, employees, and system operations. The system should provide clear notifications for actions like login, OTP verification, and updates. Users expect fast response time and secure handling of their data.

2.1.3 Environmental Requirements

The system requires a modern web browser such as Chrome, Edge, or Firefox for access. It should run on systems with standard hardware such as computers, laptops, or mobile devices. The backend requires a server environment supporting Node.js and Express.js.The database requires PostgreSQL for data storage and management. The ML service requires a Python environment with Flask and required libraries. A stable internet connection is required for communication between system components.

2.2 Design and Architecture

The system design follows a modular and distributed approach to improve maintainability, scalability, and performance. The architecture is designed using separation of concerns, where each module performs a specific function while interacting with other modules through defined interfaces. The design follows modular decomposition, dividing the system into functional modules such as Authentication, Employee Management, Patient Management, Lab Management, Prescription Management, and Ward Management. Each module is designed independently to simplify maintenance and upgrades. The architecture also follows security-driven design principles, integrating authentication mechanisms, role-based access control, OTP verification, rate limiting, and anomaly detection as part of the system design rather than as external add-ons. A RESTful API-based design is used to enable communication between the frontend and backend. This supports loose coupling between components and allows future integration with external services or microservices. The design supports scalability and extensibility, allowing future enhancements such as cloud deployment, mobile access, advanced analytics, and additional security services.

2.3 Implementation

The system is implemented using a full-stack approach, combining modern web technologies to ensure efficiency, scalability, and security. The frontend is developed using React, which provides an interactive user interface for employees, patients, and administrators. It handles user interactions, form inputs, and communicates with backend services through RESTful APIs. The backend is implemented using Node.js and Express.js, which manage business logic, API routing, authentication, and authorization. It supports JWT-based authentication for employees and session-based authentication for administrators and patients. Security features such as OTP verification, role-based access control, and rate limiting are also integrated into the backend. The database layer is implemented using PostgreSQL, which stores structured data including user credentials, patient records, lab tests, prescriptions, ward admissions, and system logs. Secure database access is ensured using parameterized queries and proper validation techniques. Additionally, a Python Flask-based machine learning service is implemented for anomaly detection. This service analyzes incoming requests and user behavior to identify suspicious activities and enhance system security. Logging and monitoring mechanisms are included to track system activities and support threat detection. Encryption techniques help protect sensitive data during storage and transmission. Error handling and input validation improve system reliability and reduce security risks. Backup and recovery mechanisms support data protection and operational continuity. The integrated implementation approach enables secure communication, coordinated processing, and efficient overall system performance. API request validation helps prevent malformed or unauthorized requests from affecting application logic. Session management controls maintain secure access across active interactions. Modular implementation improves maintainability and strengthens protection against application-level threats. Load handling mechanisms support stable performance under concurrent requests. Secure service integration improves coordination between backend and machine learning components. Audit logging supports accountability and security event analysis. Scalable architecture design allows future enhancements and feature expansion.ara

2.4 Testing

The system is tested to ensure reliability, security, and proper functionality of all components. Different levels of testing are performed to validate both individual modules and the complete system. Unit testing is conducted to verify that individual components and functions operate correctly in isolation. It tests modules such as authentication, input validation, API functions, and database operations to identify errors at an early stage. Test cases are used to confirm expected outputs, validate logic, and ensure each unit performs according to requirements. Unit testing helps detect coding issues and supports easier debugging and maintenance.

2.4.1 Unit Testing

Each module is tested independently to verify its functionality. Backend APIs, authentication logic, and database operations are tested to ensure correct outputs. This helps identify and fix errors at an early stage.

Test ID	Module	Test Scenario	Expected Result	Status
UT-01	Employee Login	Valid employee credentials	Login successful	Pass
UT-02	Employee Login	Invalid password	Error message displayed	Pass
UT-03	Admin Login	Valid admin credentials	Admin dashboard opened	Pass
UT-04	Admin Login	Invalid login attempt	Access denied	Pass
UT-05	Patient Registration	Valid patient data	Patient record created	Pass
UT-06	Patient Registration	Missing required field	Validation error shown	Pass
UT-07	OTP Verification	Valid OTP	Verification successful	Pass
UT-08	OTP Verification	Invalid OTP	Verification failed	Pass
UT-09	Password Reset	Valid token + OTP	Password updated	Pass
UT-10	Lab Module	Upload lab report	Report stored successfully	Pass
UT-11	Lab Module	Invalid report format	Upload rejected	Pass
UT-12	Prescription Module	Create prescription	Prescription saved	Pass
UT-13	Ward Admission	Admit patient	Admission record created	Pass
UT-14	Ward Discharge	Discharge patient	Status updated	Pass
UT-15	JWT Validation	Expired token	Access denied	Pass

2.4.2 Integration Testing

Integration testing verifies communication between system modules such as frontend, backend, database, and ML service.

Test ID	Integrated Modules	Test Scenario	Expected Result	Status
IT-01	Frontend + Backend	Employee login request	Authentication successful	Pass
IT-02	Frontend + Backend	Admin login request	Admin access granted	Pass
IT-03	Frontend + Backend + Database	Patient registration	Data stored correctly	Pass
IT-04	Backend + Database	Employee profile retrieval	Profile returned successfully	Pass
IT-05	Backend + Email Server	OTP generation request	OTP delivered successfully	Pass
IT-06	Backend + Email Server	Password reset email	Reset email sent	Pass
IT-07	Backend + ML Service	Suspicious login detection	Threat identified	Pass
IT-08	Backend + ML Service	Normal request analysis	Request allowed	Pass
IT-09	Backend + Database	Prescription creation	Record stored correctly	Pass
IT-10	Backend + Database	Retrieve patient prescriptions	Data fetched successfully	Pass
IT-11	Backend + Database	Ward admission request	Admission record created	Pass
IT-12	Backend + Database	Patient discharge update	Status updated	Pass
IT-13	Frontend + Backend + Database	Lab test request retrieval	Requests displayed	Pass
IT-14	Backend + Database	Lab report upload	Report stored successfully	Pass
IT-15	Backend + Security Middleware	JWT validation request	Access verified	Pass

IT-16	Backend + Session Middleware	Patient session validation	Session active	Pass
IT-17	Frontend + Backend + ML Service	Suspicious activity blocked	Access denied	Pass
IT-18	Backend + Logging Service	Security event logging	Log recorded	Pass
IT-19	Backend + Database + Email	New employee creation	Record created and email sent	Pass
IT-20	Full Workflow Integration	Login → Patient → Lab → Prescription	Complete flow successful	Pass

2.4.3 System Testing

The complete system is tested as a whole to verify all features work together. It includes workflows such as login, registration, lab operations, and prescriptions.

Test ID	System Scenario	Test Description	Expected Result	Status
ST-01	Employee Login	Login with valid employee credentials	Dashboard displayed	Pass
ST-02	Employee Login Failure	Login with invalid credentials	Error shown	Pass
ST-03	Admin Login Workflow	Admin login and dashboard access	Access granted	Pass
ST-04	Patient Registration	Register a new patient	Registration completed	Pass
ST-05	Patient Session Access	Patient login and session creation	Session active	Pass
ST-06	OTP Verification	OTP verification process	Verification successful	Pass
ST-07	Password Reset Flow	Reset using token and OTP	Password updated	Pass

ST-08	Employee-Patient Access	Employee retrieves assigned patients	Patient list displayed	Pass
ST-09	Lab Request Workflow	View lab requests	Requests displayed	Pass
ST-10	Lab Completion Flow	Mark lab request complete	Status updated	Pass
ST-11	Lab Report Upload	Upload lab test report	Report stored	Pass
ST-12	Prescription Creation	Create prescription	Prescription saved	Pass
ST-13	Prescription Retrieval	View patient prescriptions	Data retrieved	Pass
ST-14	Ward Admission	Admit patient to ward	Admission recorded	Pass
ST-15	Ward Discharge	Discharge admitted patient	Status updated	Pass
ST-16	Suspicious Login Detection	Abnormal request sent	Request blocked	Pass
ST-17	Role-Based Access	Unauthorized module access attempt	Access denied	Pass
ST-18	End-to-End Patient Workflow	Register → Lab → Prescription	Full workflow completed	Pass
ST-19	End-to-End Admin Workflow	Admin login → Manage employee	Workflow completed	Pass
ST-20	Full System Workflow	Login → Patient → Lab → Prescription → Discharge	Complete flow successful	Pass

2.4.4 Security Testing

The system is tested for vulnerabilities such as SQL injection, XSS, brute-force attacks, and unauthorized access.

Test ID	Security Scenario	Test Input / Action	Expected Result	Status
SEC-01	SQL Injection Attempt	Inject SQL in login field	Request blocked	Pass
SEC-02	SQL Injection on API	Malicious query in API parameter	Input rejected	Pass
SEC-03	XSS Attack Attempt	Script injected in form field	Script sanitized/blocked	Pass
SEC-04	Stored XSS Attempt	Malicious payload saved attempt	Data rejected	Pass
SEC-05	Brute Force Login	Multiple failed logins	IP/User temporarily blocked	Pass
SEC-06	Invalid JWT Token	Modified token used	Access denied	Pass
SEC-07	Expired JWT Token	Expired token request	Session rejected	Pass
SEC-08	Unauthorized API Access	Protected endpoint without auth	401/403 returned	Pass
SEC-09	Role Privilege Escalation	Employee attempts admin action	Access denied	Pass
SEC-10	Session Hijacking Attempt	Invalid session reuse	Session terminated	Pass
SEC-11	Invalid OTP Submission	Wrong OTP entered	Verification failed	Pass
SEC-12	OTP Replay Attempt	Reused OTP	Request rejected	Pass
SEC-13	Suspicious Request Pattern	Abnormal request frequency	ML anomaly detected	Pass
SEC-14	Bot Request Simulation	Automated repeated requests	Request blocked	Pass

SEC-15	Rate Limit Exceeded	Excessive API calls	Requests throttled	Pass
SEC-16	Password Hash Protection	Credential storage review	Passwords hashed securely	Pass
SEC-17	CSRF Attempt	Forged request submission	Request denied	Pass
SEC-18	Invalid File Upload	Malicious lab file upload	Upload rejected	Pass
SEC-19	Security Header Validation	Header inspection	Headers correctly set	Pass
SEC-20	Audit Log Verification	Security event triggered	Event logged	Pass

2.4.5 Performance Testing

Performance testing validates response time, load handling, and scalability.

Test ID	Performance Parameter	Test Condition	Result	Status
PT-01	Login Response Time	Single user login	< 2 sec	Pass
PT-02	API Response Time	Standard API request	< 1.5 sec	Pass
PT-03	Database Query Time	Patient record retrieval	< 500 ms	Pass
PT-04	Concurrent Users	50 simultaneous users	Stable	Pass
PT-05	Concurrent Users	100 simultaneous users	Stable	Pass
PT-06	Concurrent Users	200 simultaneous users	Acceptable	Pass
PT-07	Lab Report Upload Time	Standard file upload	< 3 sec	Pass
PT-	Prescription Save Time	Create prescription	< 1 sec	Pass

08				
PT-09	ML Detection Response	Suspicious request analysis	< 1 sec	Pass
PT-10	OTP Delivery Time	Send OTP email	< 5 sec	Pass
PT-11	Session Validation Time	Active session check	< 500 ms	Pass
PT-12	Large Data Retrieval	1000 records query	Acceptable	Pass
PT-13	CPU Utilization	Normal load	Within limit	Pass
PT-14	Memory Usage	Normal load	Stable	Pass
PT-15	Peak Load Stability	High request volume	Stable	Pass
PT-16	Error Rate	Under normal traffic	Minimal	Pass
PT-17	Throughput	Requests processed/sec	Acceptable	Pass
PT-18	Server Recovery	Temporary overload	Recovered	Pass
PT-19	Security Check Latency	JWT + ML validation	Acceptable	Pass
PT-20	End-to-End Workflow Time	Login to discharge flow	Acceptable	Pass

2.5 Graphical User Interface (GUI) Layout

The Graphical User Interface (GUI) of the system is designed to be simple, user-friendly, and responsive, ensuring ease of use for employees, patients, and administrators. The interface is developed using React and provides structured navigation for accessing different system functionalities.

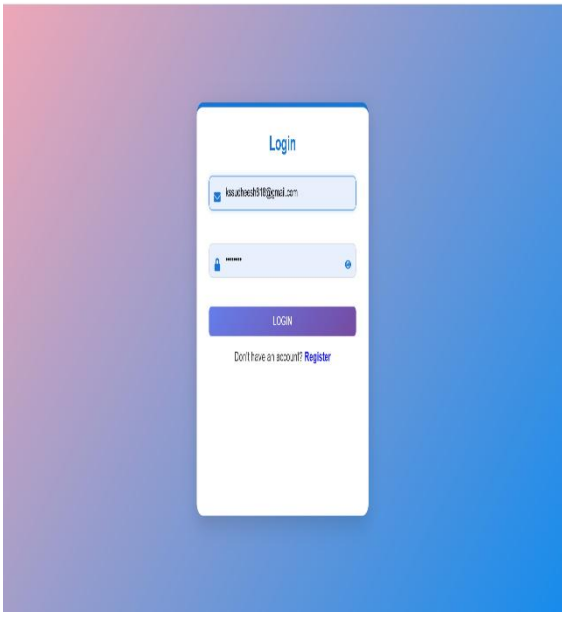


Figure 2.5.3: Patient Login Page

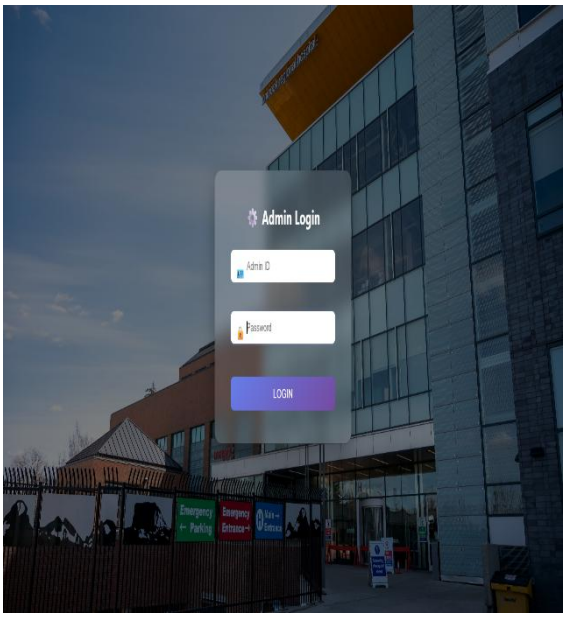


Figure 2.5.1: Admin Login Page

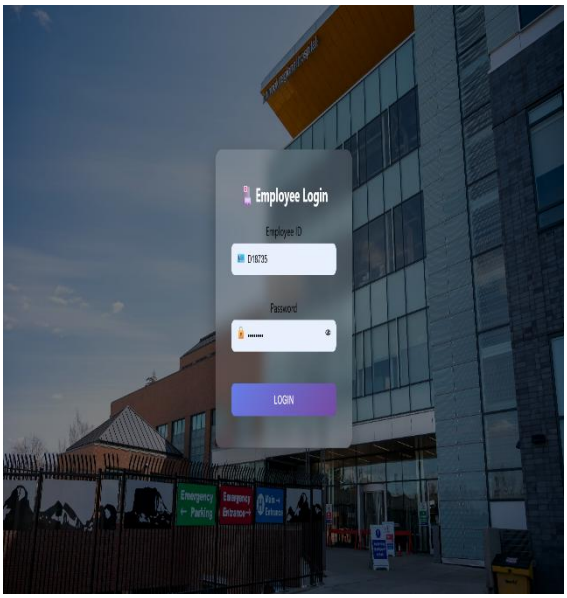


Figure 2.5.3: Employee Login Page

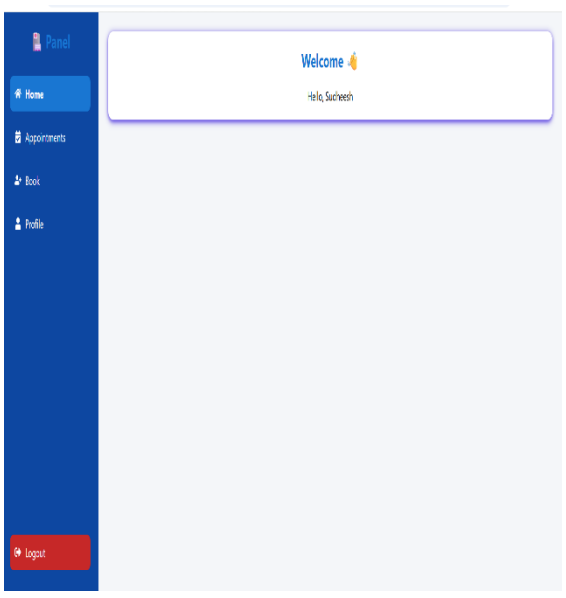


Figure 2.5.3: Patient Dashboard

2.6 Customer Testing

Customer testing is conducted to evaluate the system from the end-user perspective and ensure that it meets user requirements and expectations. This phase involves real users such as employees, administrators, and patients interacting with the system in a controlled environment. Users are asked to perform common tasks such as login, patient registration, lab test updates, and viewing prescriptions. Their feedback is collected regarding usability, performance, and overall satisfaction. This helps identify issues related to user experience and functionality. Based on the feedback received, necessary improvements are made to enhance system usability, interface design, and performance. Customer testing ensures that the system is user-friendly, reliable, and ready for real-world deployment.

2.7 Evaluation

The system is evaluated to assess its overall performance, security, and effectiveness in meeting the project objectives, focusing on functionality, usability, security, and system reliability. Functionally, the system supports core operations including authentication, patient management, lab processing, prescriptions, and ward management, with all modules integrated to ensure smooth workflow across the application. From a usability perspective, the system provides a user-friendly interface with clear navigation, enabling employees, patients, and administrators to perform tasks efficiently, while satisfactory response time supports a smooth user experience. In terms of security, the system performs effectively through JWT authentication, session management, OTP verification, and ML-based anomaly detection, which help protect against unauthorized access and common cyber threats. System reliability is supported through stable performance and consistent

operation across modules. Error handling and validation mechanisms contribute to accurate processing and reduced failures. Logging and monitoring support performance evaluation and identification of security events. Overall, the evaluation shows the system meets functional, security, and operational requirements effectively.

Evaluation Results

Parameter	**Result**
Login Response Time	< 2 seconds
API Response Time	< 1.5 seconds
Database Query Time	< 500 ms
Suspicious Requests Blocked	Successful
Authentication Validation	Successful
Role-Based Access Control	Implemented
System Availability	High
Data Integrity	Maintained

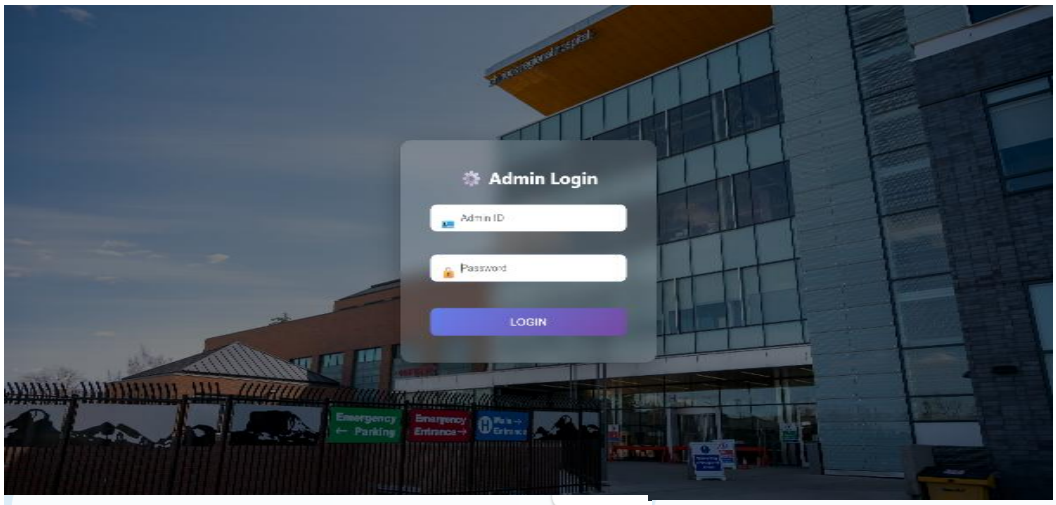
Traditional System vs Proposed System

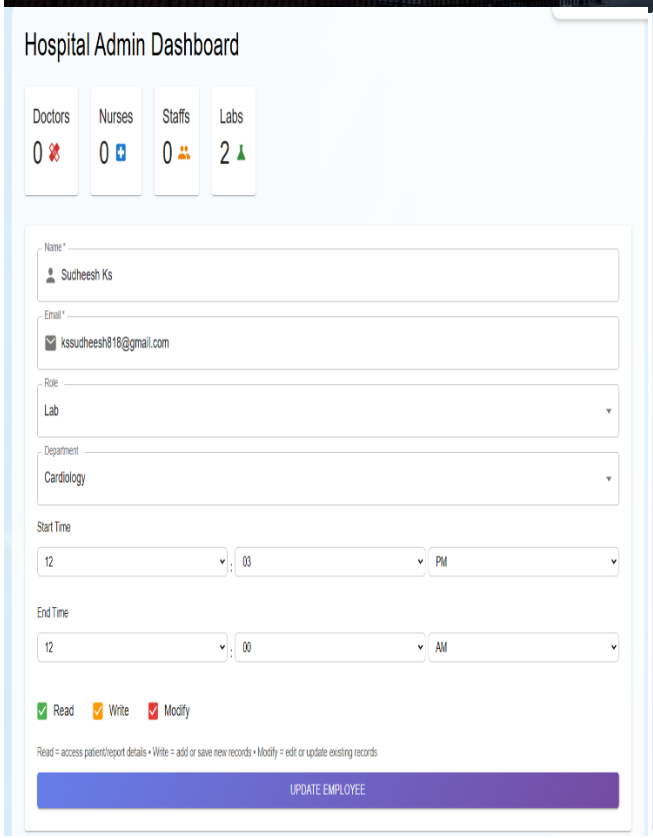
Parameter	**Traditional System**	**Proposed System**
Authentication	Basic Login	JWT + Session + OTP
Security Monitoring	Manual	ML-Based Detection

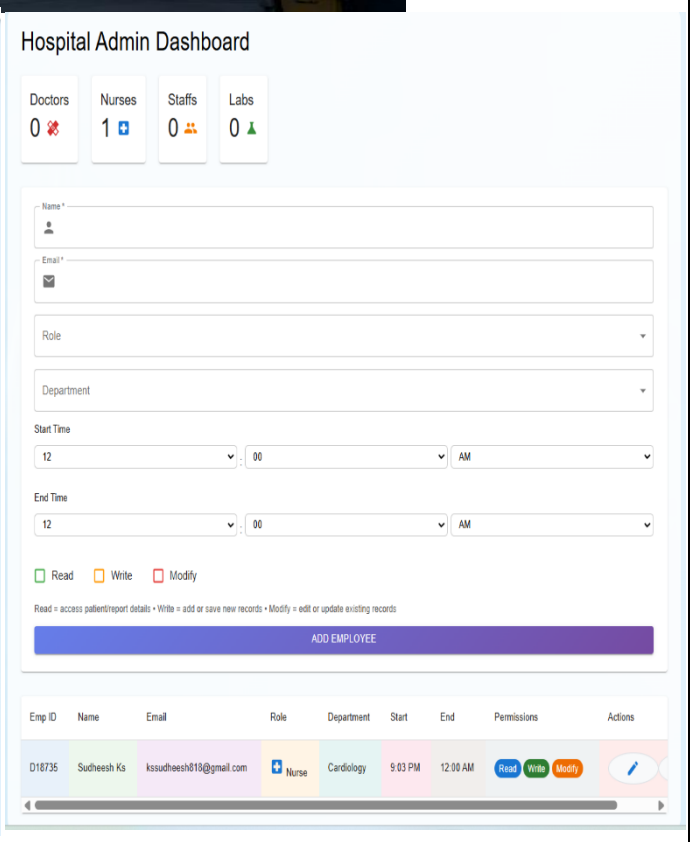
Access Control	Limited	Role-Based Access
Threat Detection	Not Available	Automated
Data Management	Semi-Manual	Centralized Database
Scalability	Limited	High
Response Time	Moderate	Faster
Reliability	Lower	Higher

3 Snapshots of the Project

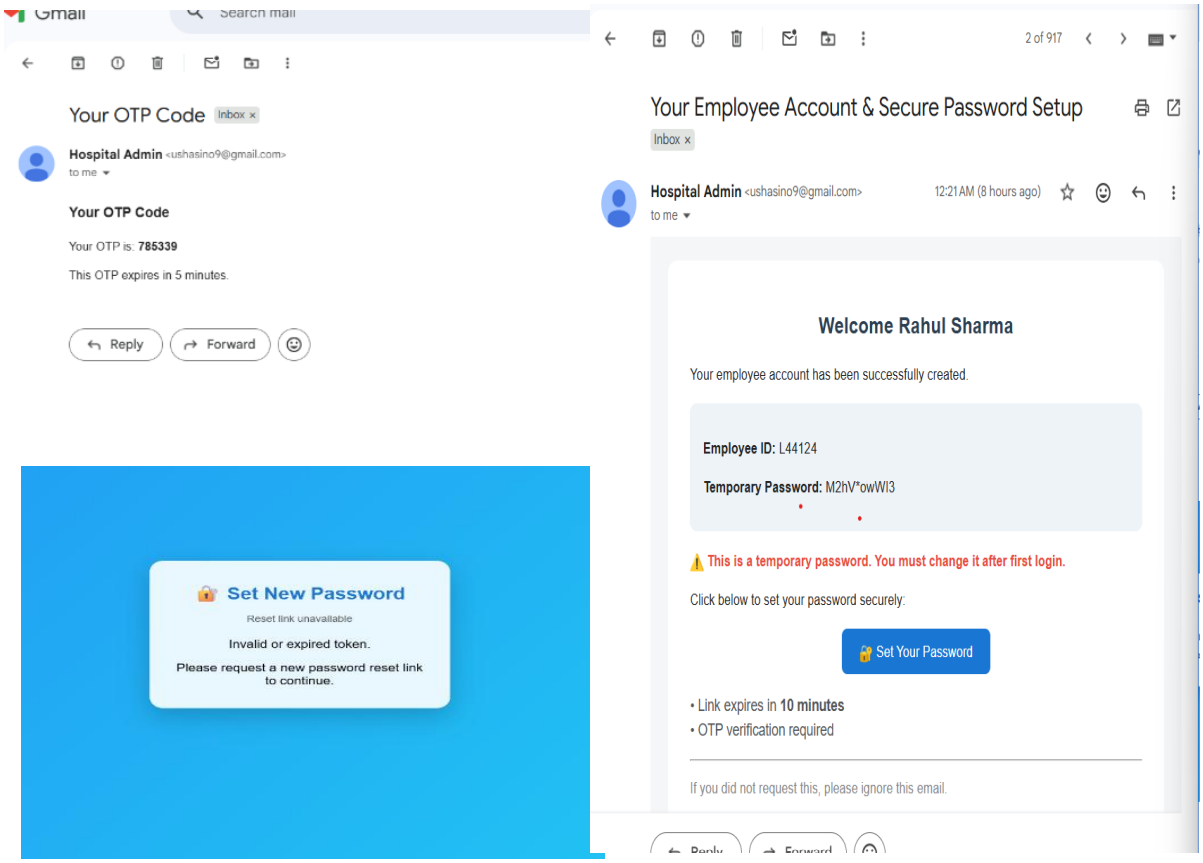
- Admin Dashboard

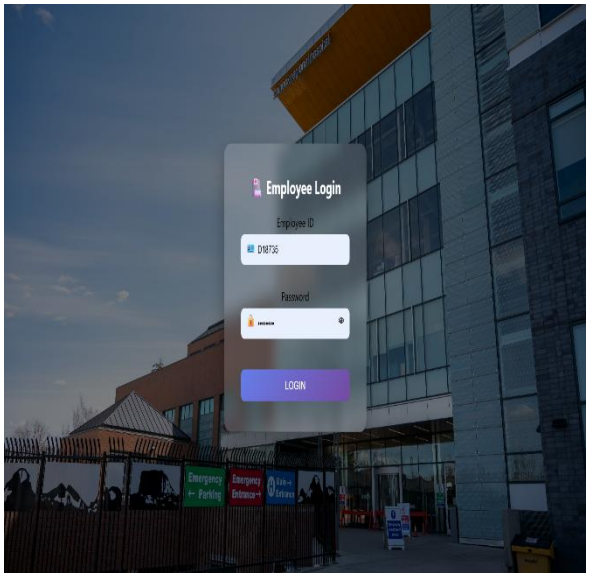




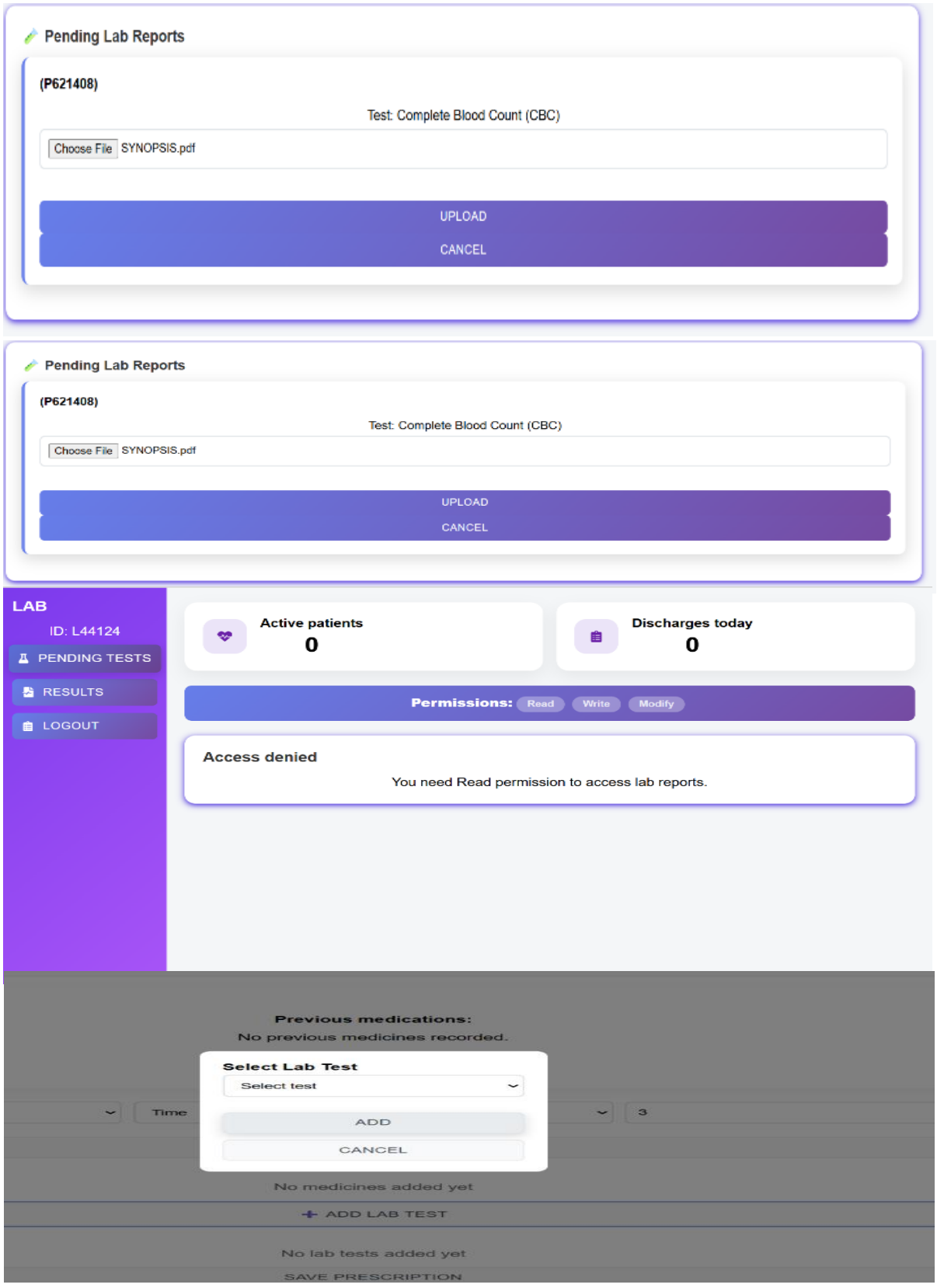


- Employee Panel

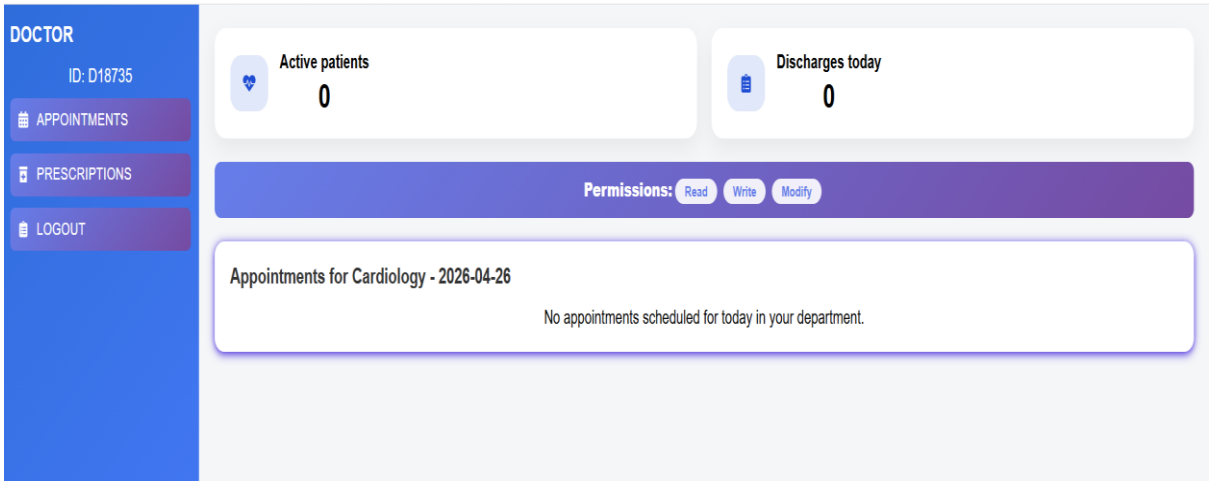


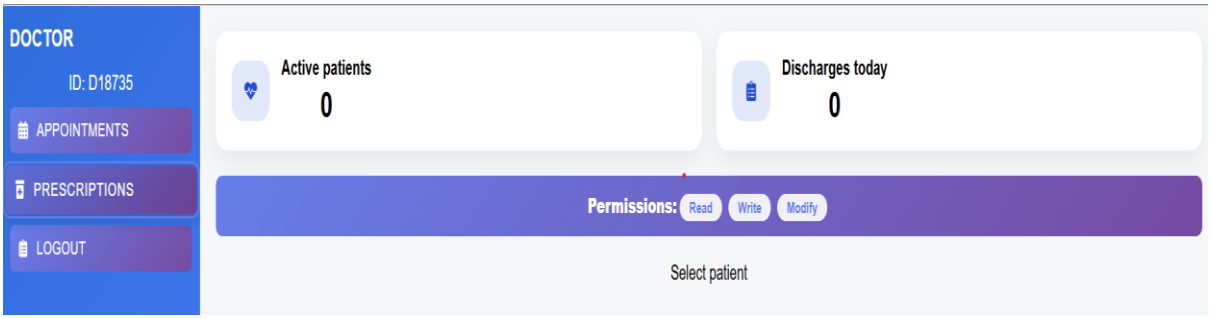


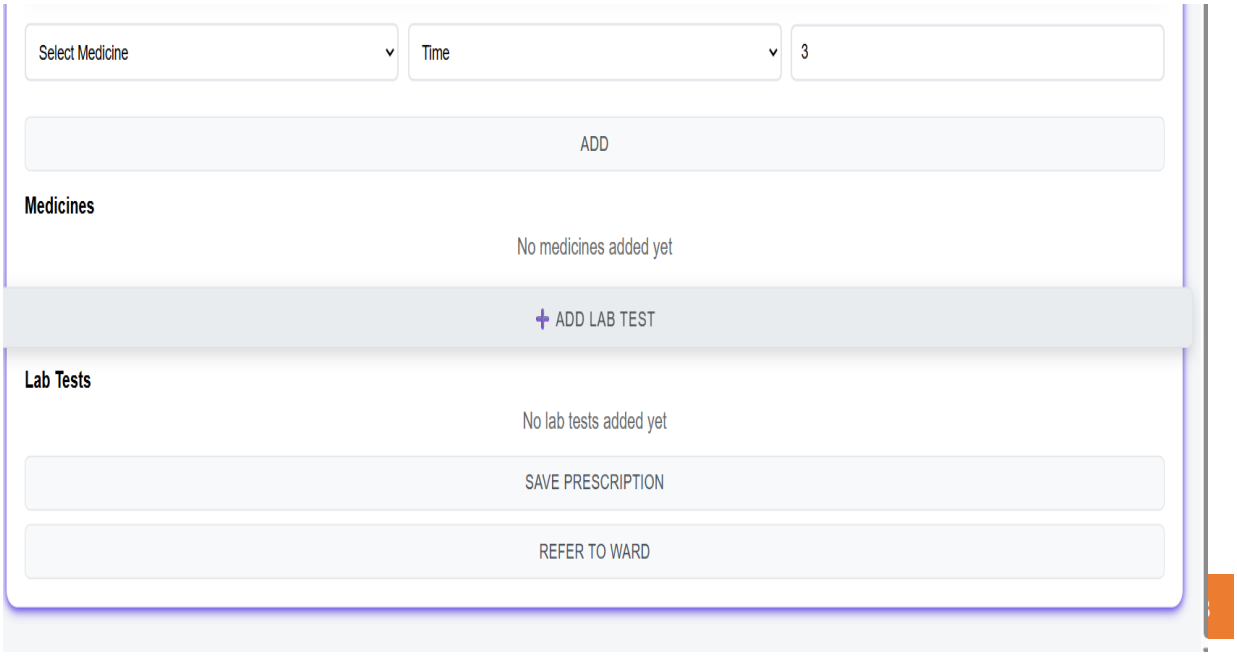
❖ LAB

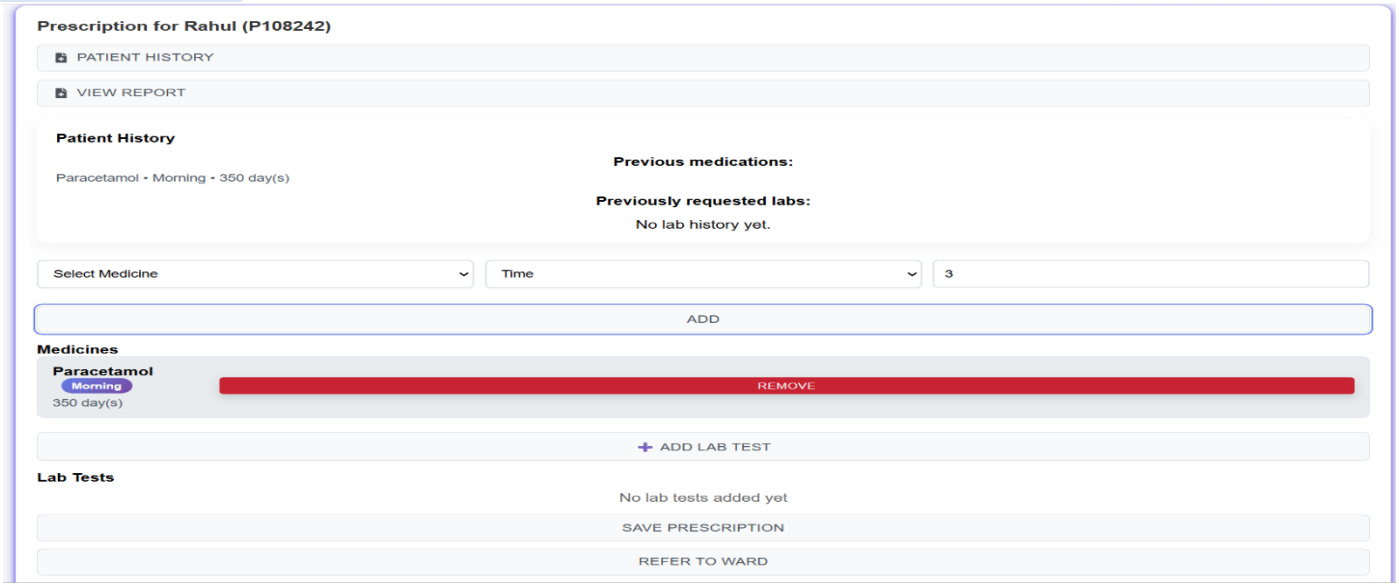

Pending Lab Reports
(P621408)
Test: Complete Blood Count (CBC)
Choose File SYNOPSIS.pdf
UPLOAD
CANCEL
Pending Lab Reports
(P621408)
Test: Complete Blood Count (CBC)
Choose File SYNOPSIS.pdf
UPLOAD
CANCEL
LAB
ID: L44124
PENDING TESTS
RESULTS
LOGOUT
Active patients
0
Discharges today
0
Permissions: Read Write Modify
Access denied
You need Read permission to access lab reports.
Previous medications:
No previous medicines recorded.
Select Lab Test
Select test
ADD
CANCEL
Time
3
No medicines added yet
ADD LAB TEST
No lab tests added yet
SAVE PRESCRIPTION

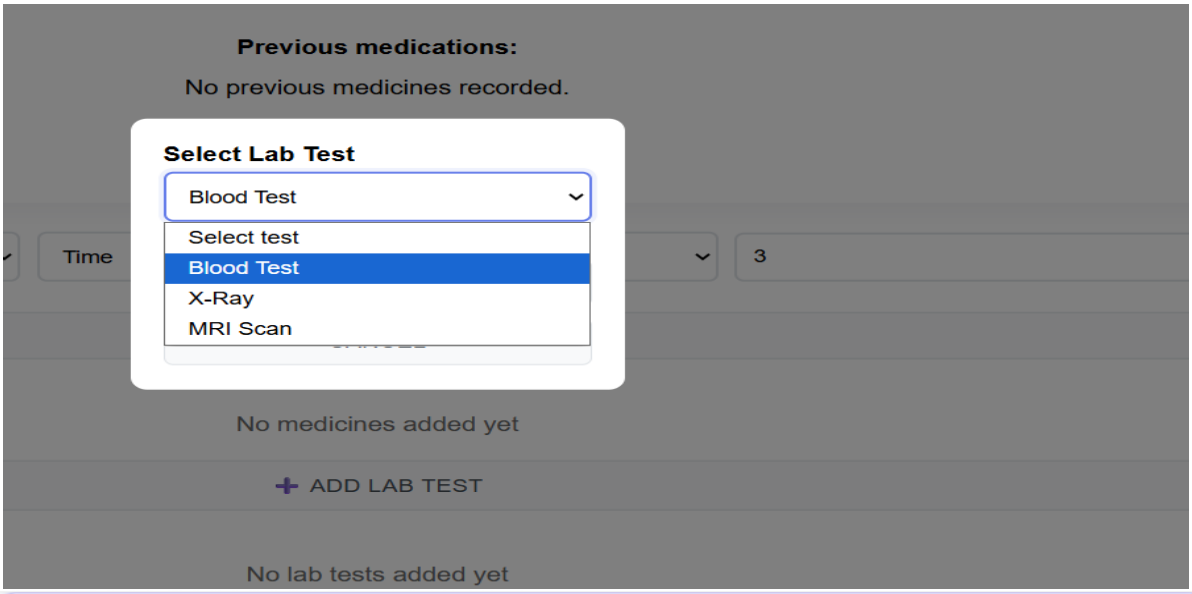
❖ DOCTOR

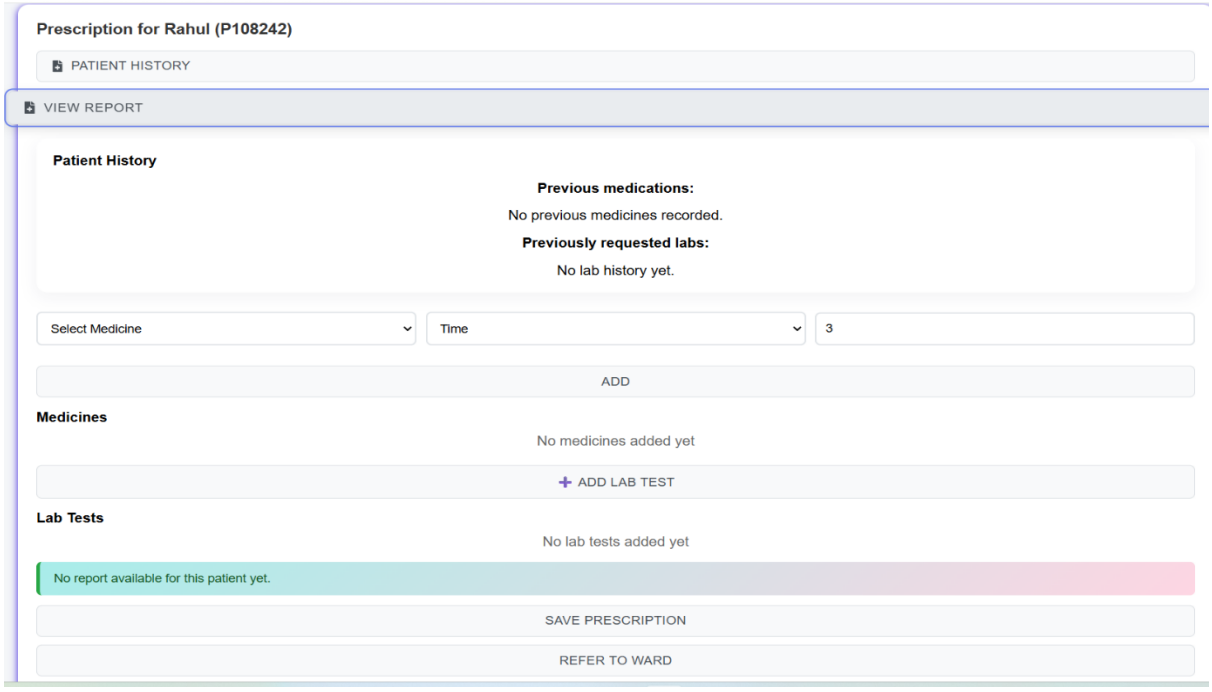

DOCTOR
ID: D18735
APPOINTMENTS
PRESCRIPTIONS
LOGOUT
Active patients
0
Discharges today
0
Permissions: Read Write Modify
Appointments for Cardiology - 2026-04-26
No appointments scheduled for today in your department.

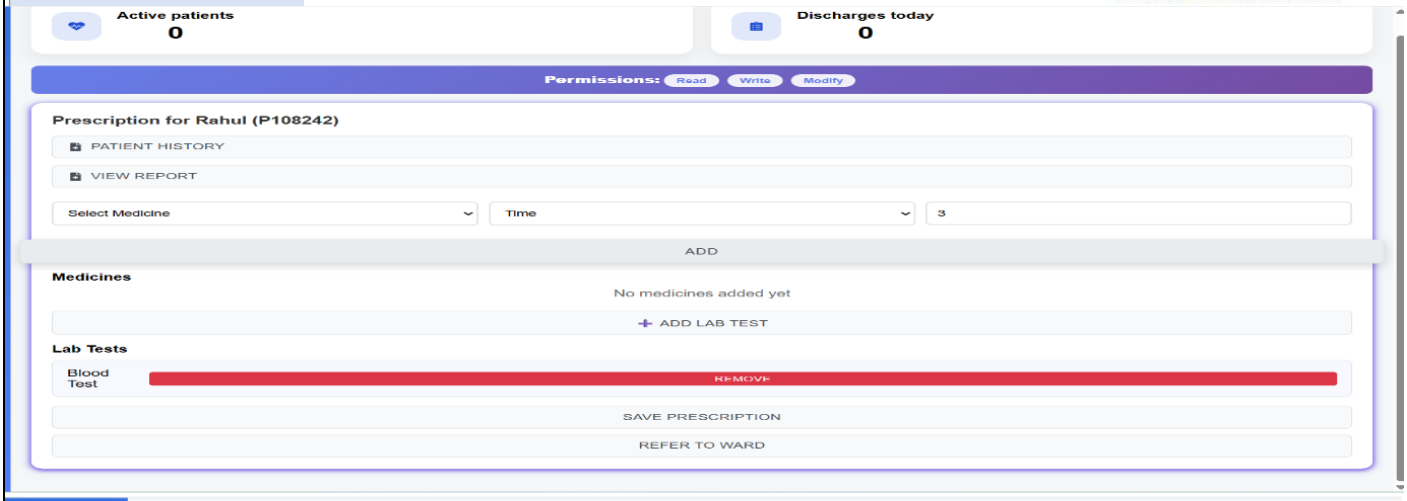

DOCTOR
ID: D18735
APPOINTMENTS
PRESCRIPTIONS
LOGOUT
Active patients
0
Discharges today
0
Permissions: Read Write Modify
Select patient

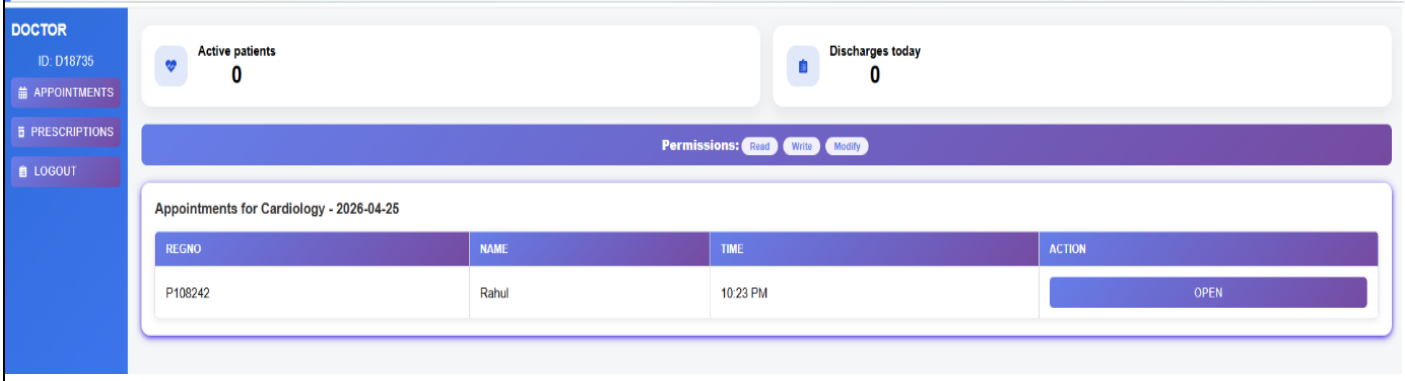

Select Medicine
Time
3
ADD
Medicines
No medicines added yet
ADD LAB TEST
Lab Tests
No lab tests added yet
SAVE PRESCRIPTION
REFER TO WARD

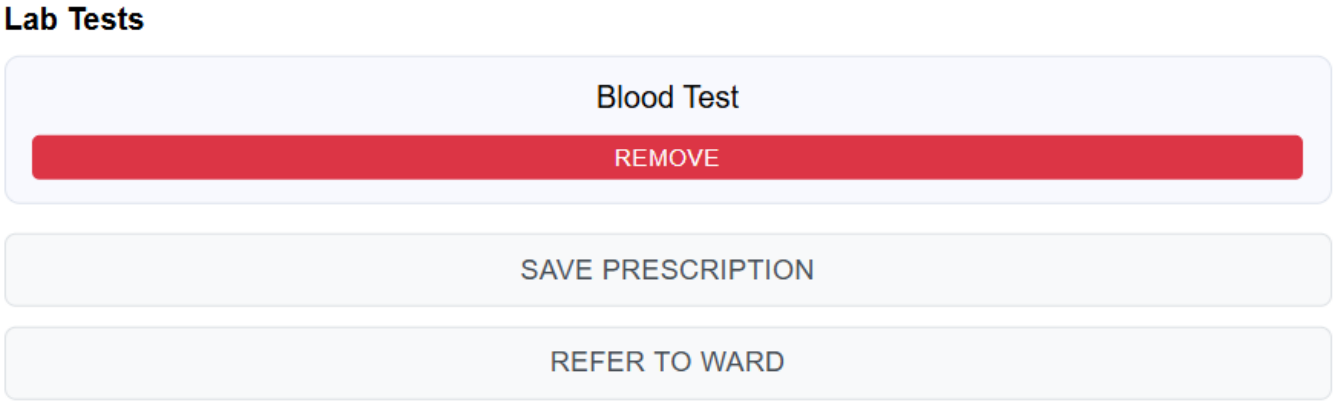

Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Patient History
Previous medications:
Paracetamol • Morning • 350 day(s)
Previously requested labs:
No lab history yet.
Select Medicine
Time
3
ADD
Medicines
Paracetamol
Morning
350 day(s)
REMOVE
ADD LAB TEST
Lab Tests
No lab tests added yet
SAVE PRESCRIPTION
REFER TO WARD

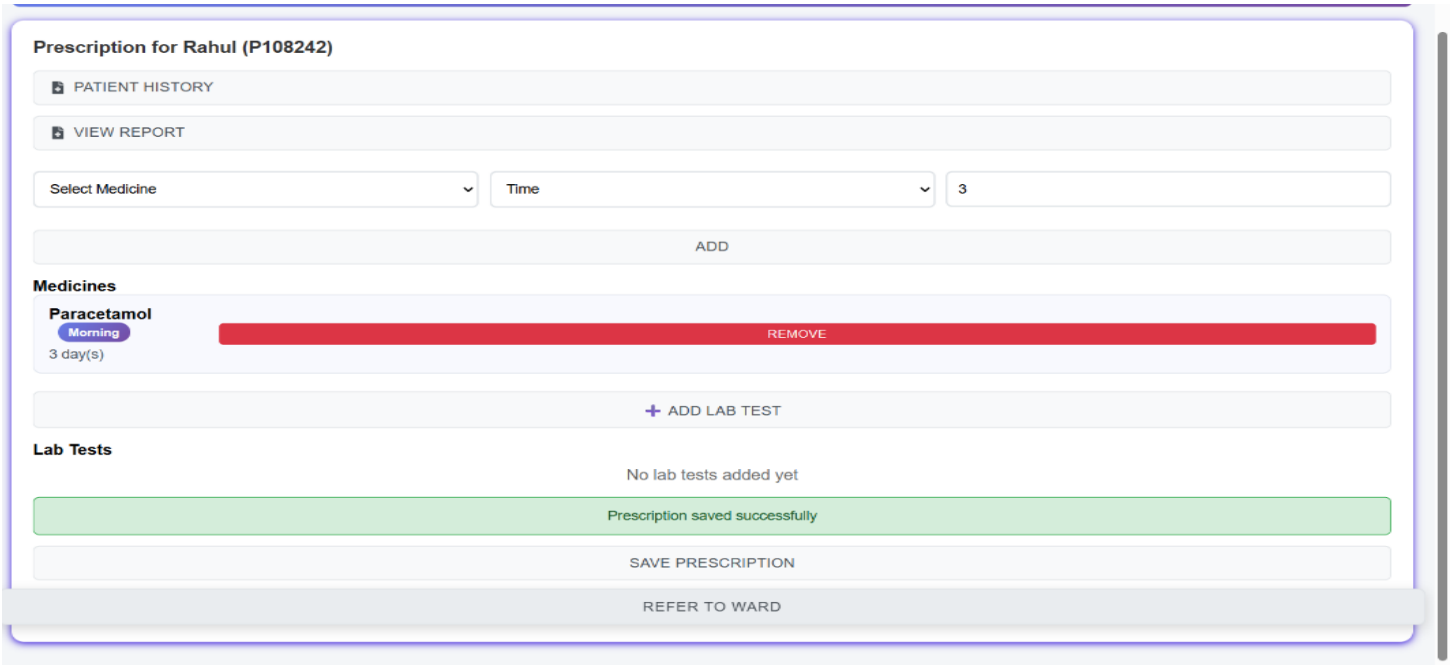

Previous medications:
No previous medicines recorded.
Select Lab Test
Blood Test
Select test
Blood Test
X-Ray
MRI Scan
Time
3
No medicines added yet
ADD LAB TEST
No lab tests added yet

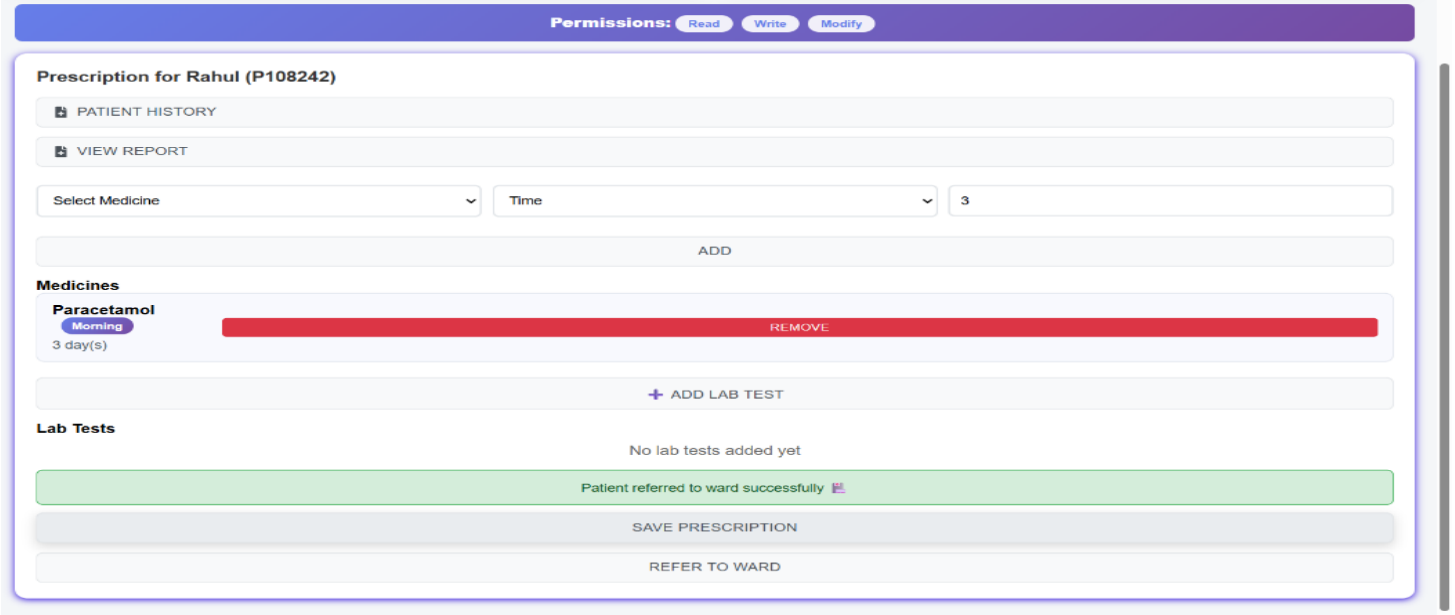

Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Patient History
Previous medications:
No previous medicines recorded.
Previously requested labs:
No lab history yet.
Select Medicine
Time
3
ADD
Medicines
No medicines added yet
ADD LAB TEST
Lab Tests
No lab tests added yet
No report available for this patient yet.
SAVE PRESCRIPTION
REFER TO WARD

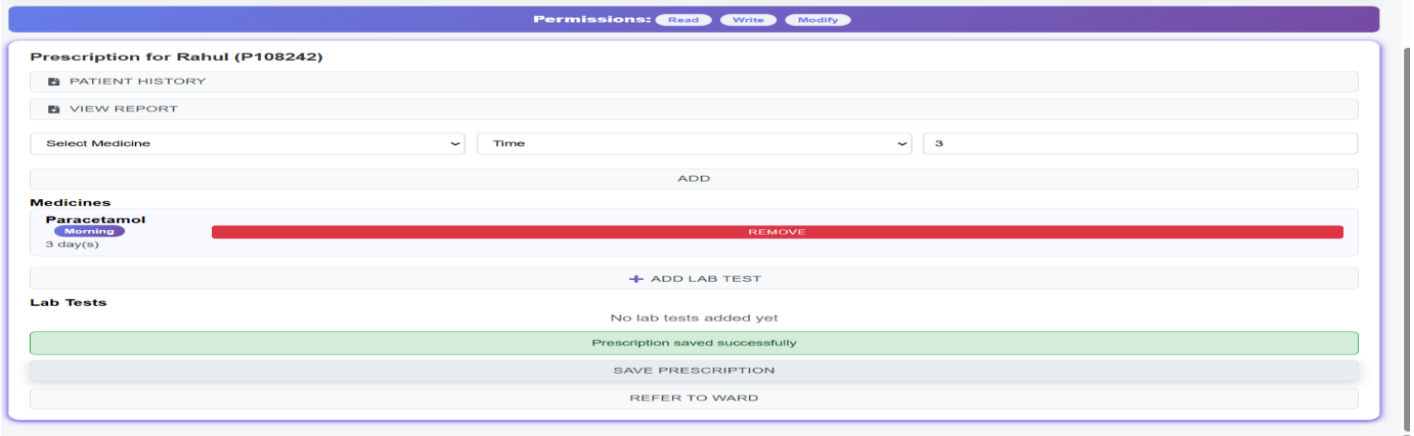

Active patients
0
Discharges today
0
Permissions: Read Write Modify
Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Select Medicine
Time
3
ADD
Medicines
No medicines added yet
ADD LAB TEST
Lab Tests
Blood Test
REMOVE
SAVE PRESCRIPTION
REFER TO WARD


DOCTOR
ID: D18735
APPOINTMENTS
PRESCRIPTIONS
LOGOUT
Active patients
0
Discharges today
0
Permissions: Read Write Modify
Appointments for Cardiology - 2026-04-25
REGNO
NAME
TIME
ACTION
P108242
Rahul
10:23 PM
OPEN

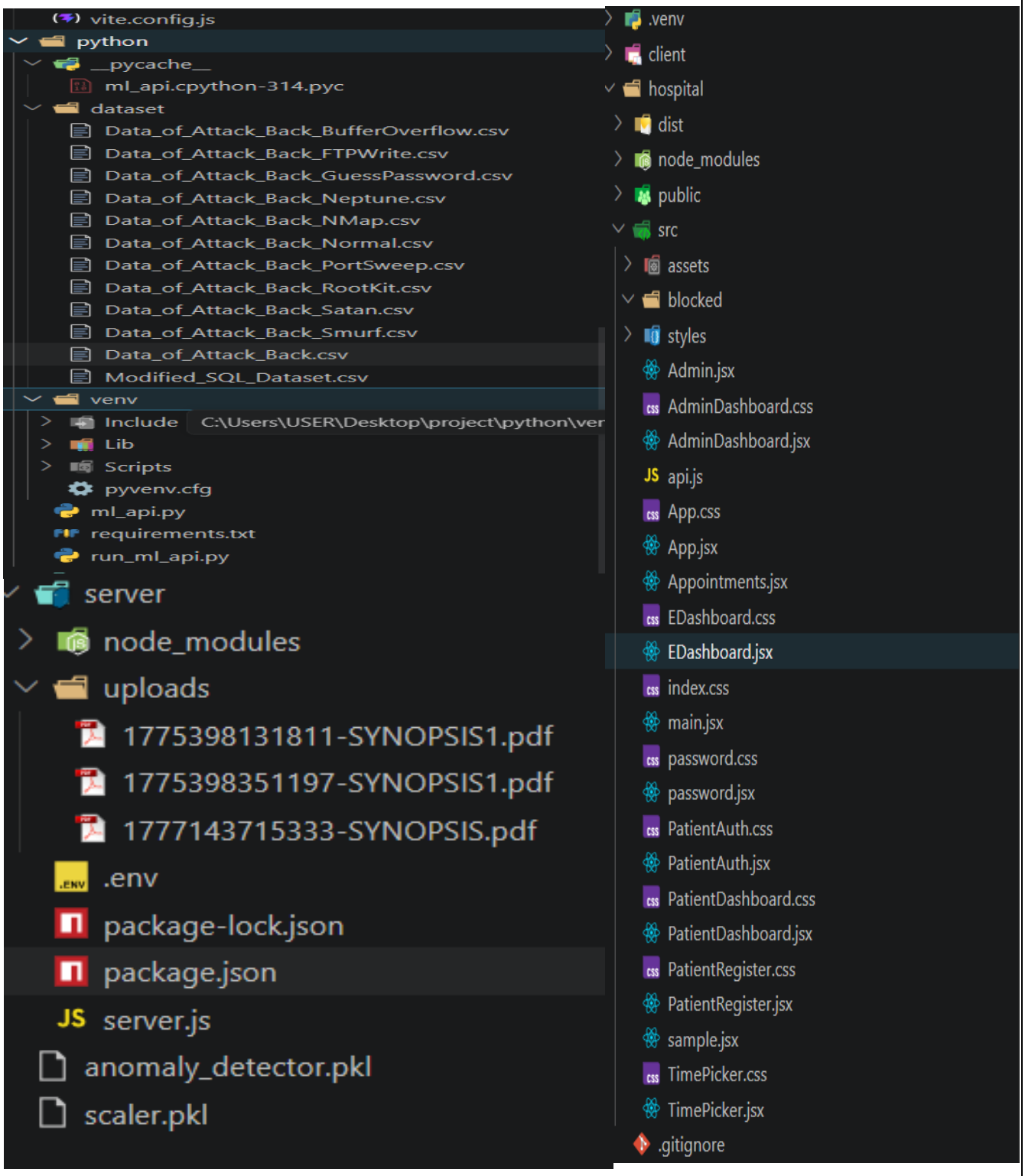

Lab Tests
Blood Test
REMOVE
SAVE PRESCRIPTION
REFER TO WARD


Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Select Medicine
Time
3
ADD
Medicines
Paracetamol
Morning
3 day(s)
REMOVE
ADD LAB TEST
Lab Tests
No lab tests added yet
Prescription saved successfully
SAVE PRESCRIPTION
REFER TO WARD

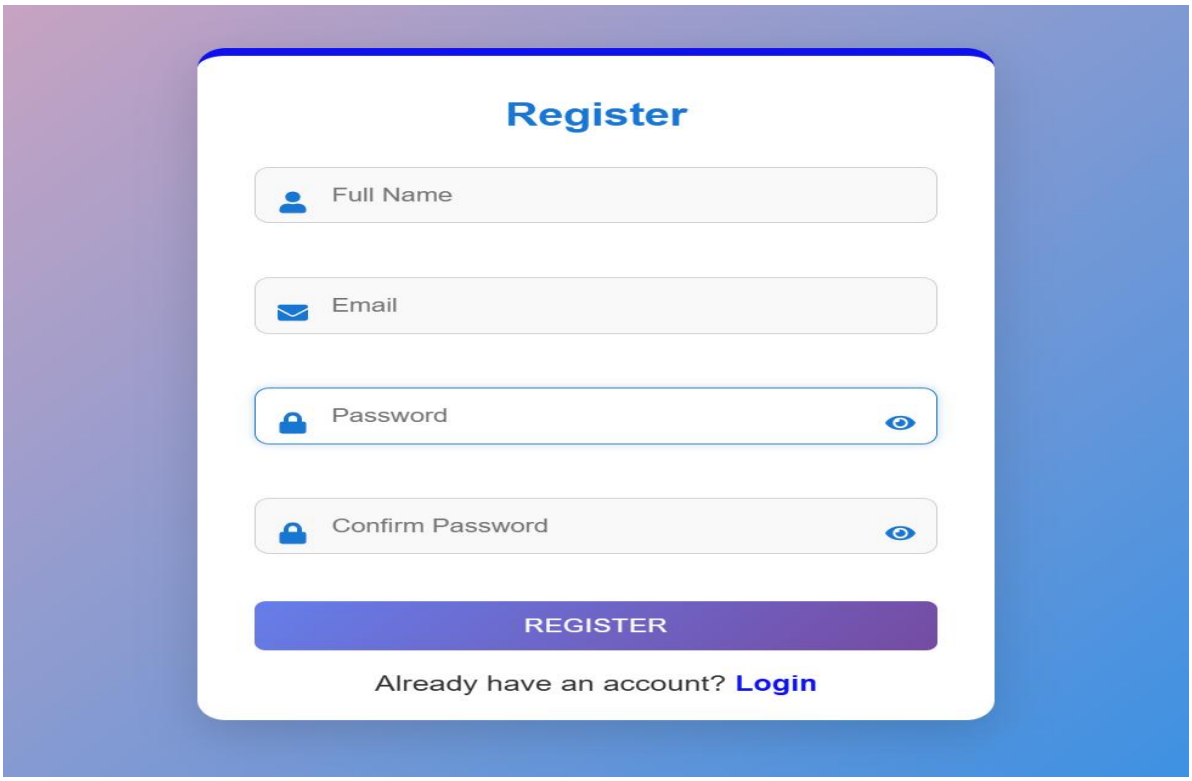

Permissions: Read Write Modify
Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Select Medicine
Time
3
ADD
Medicines
Paracetamol
Morning
3 day(s)
REMOVE
ADD LAB TEST
Lab Tests
No lab tests added yet
Patient referred to ward successfully
SAVE PRESCRIPTION
REFER TO WARD

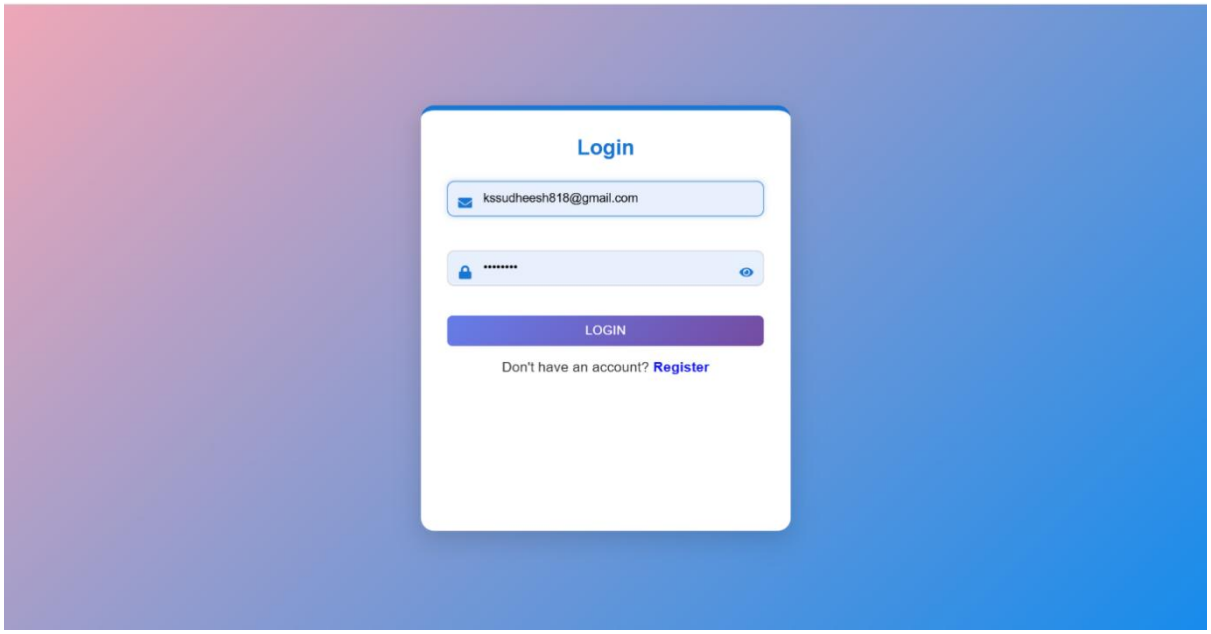

Permissions: Read Write Modify
Prescription for Rahul (P108242)
PATIENT HISTORY
VIEW REPORT
Select Medicine
Time
3
ADD
Medicines
Paracetamol
Morning
3 day(s)
REMOVE
ADD LAB TEST
Lab Tests
No lab tests added yet
Prescription saved successfully
SAVE PRESCRIPTION
REFER TO WARD

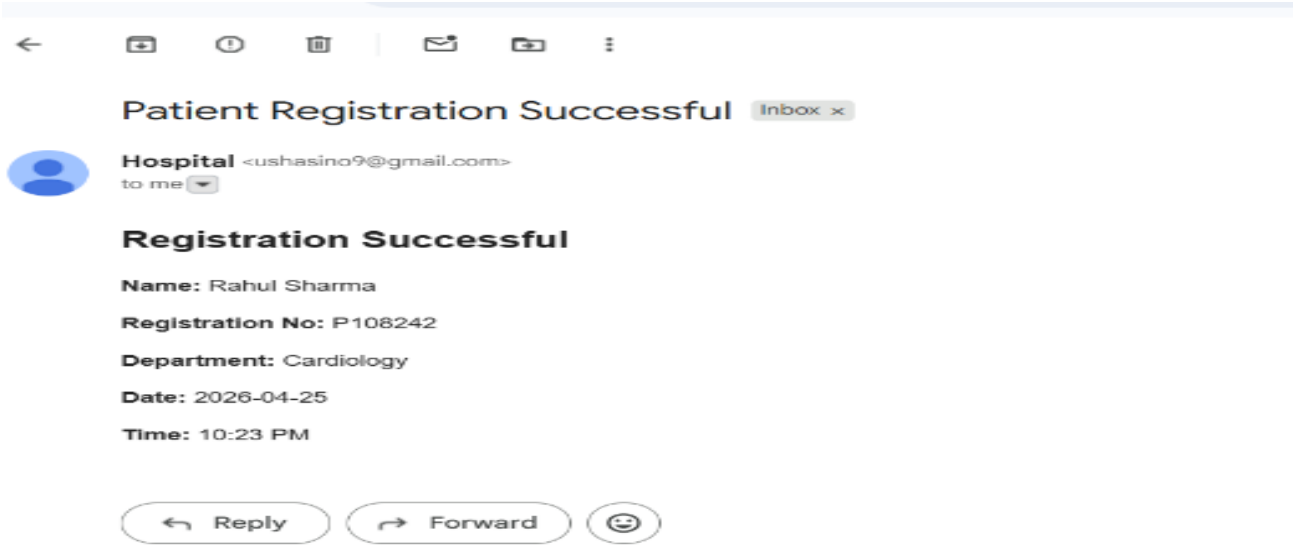
➢ Folder Structure



- Patient Interface







Patient Registration Successful Inbox ×

Hospital <ushasino9@gmail.com>
to me

Registration Successful

Name: Rahul Sharma

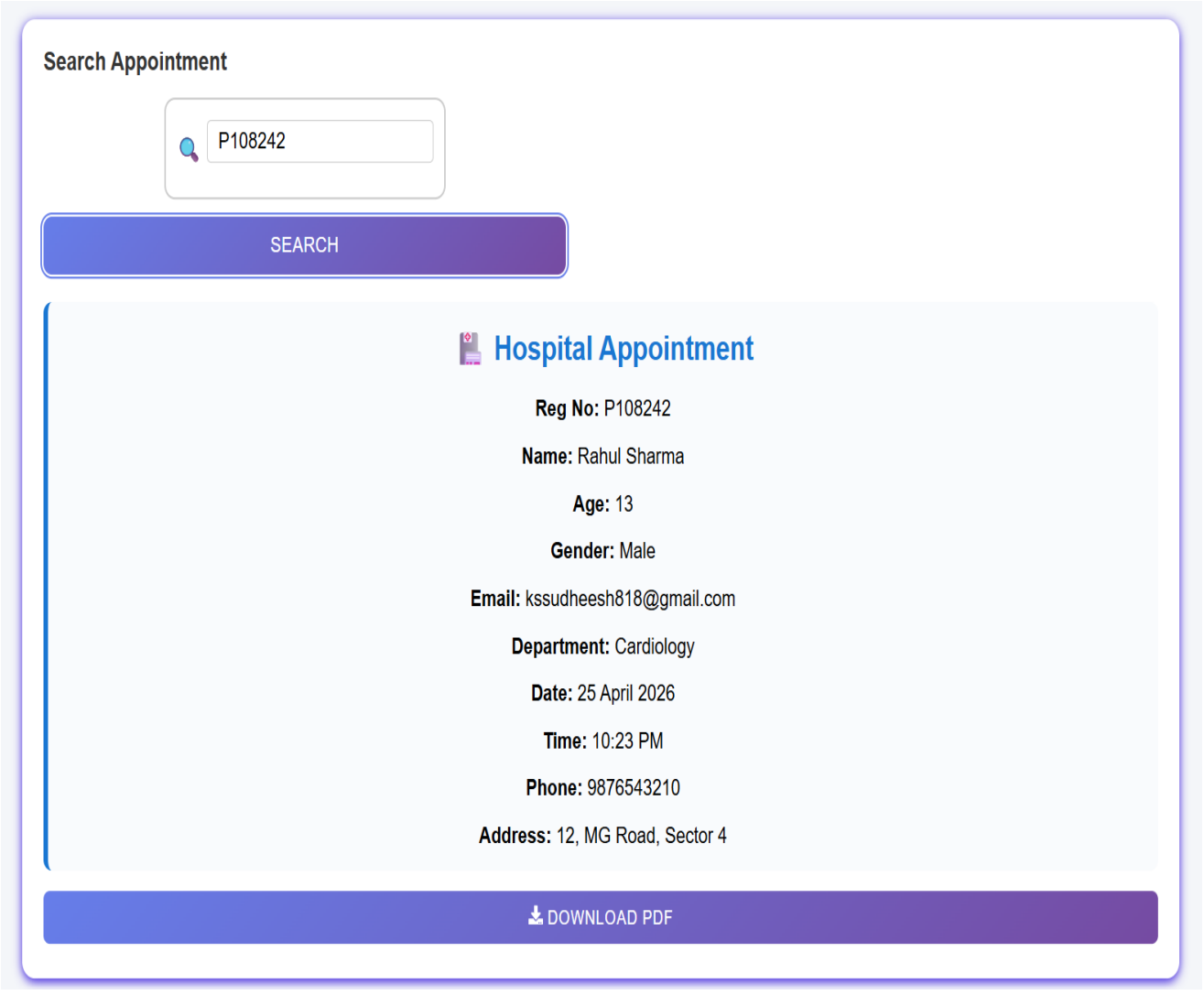
Registration No: P108242

Department: Cardiology

Date: 2026-04-25

Time: 10:23 PM

Reply Forward



Search Appointment

P108242

SEARCH

Hospital Appointment

Reg No: P108242

Name: Rahul Sharma

Age: 13

Gender: Male

Email: kssudheesh818@gmail.com

Department: Cardiology

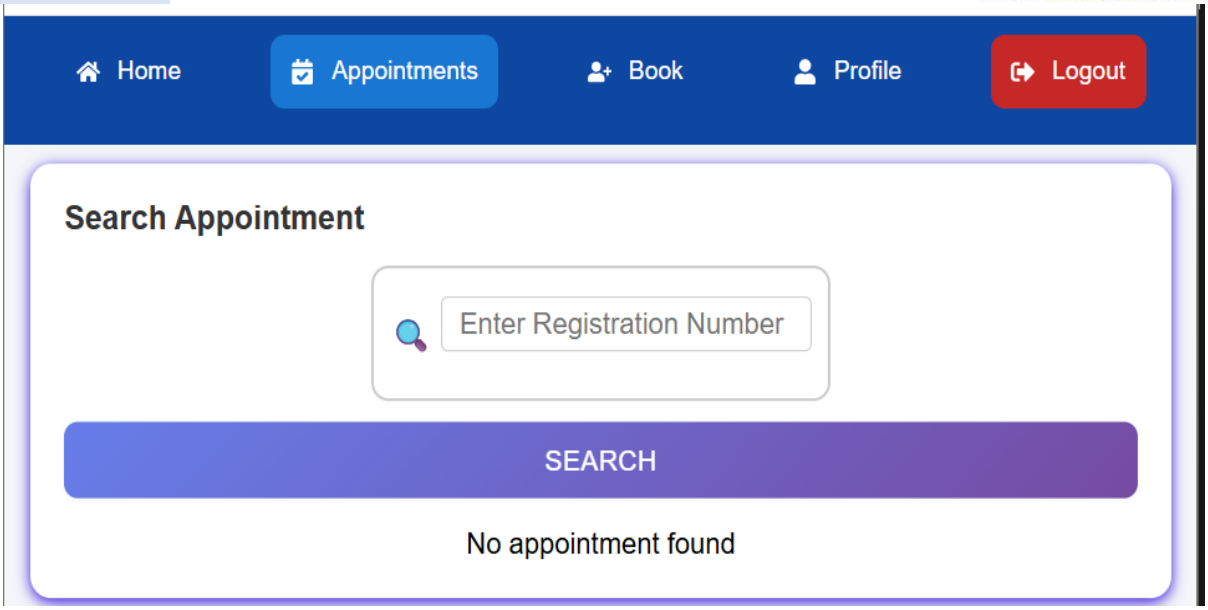
Date: 25 April 2026

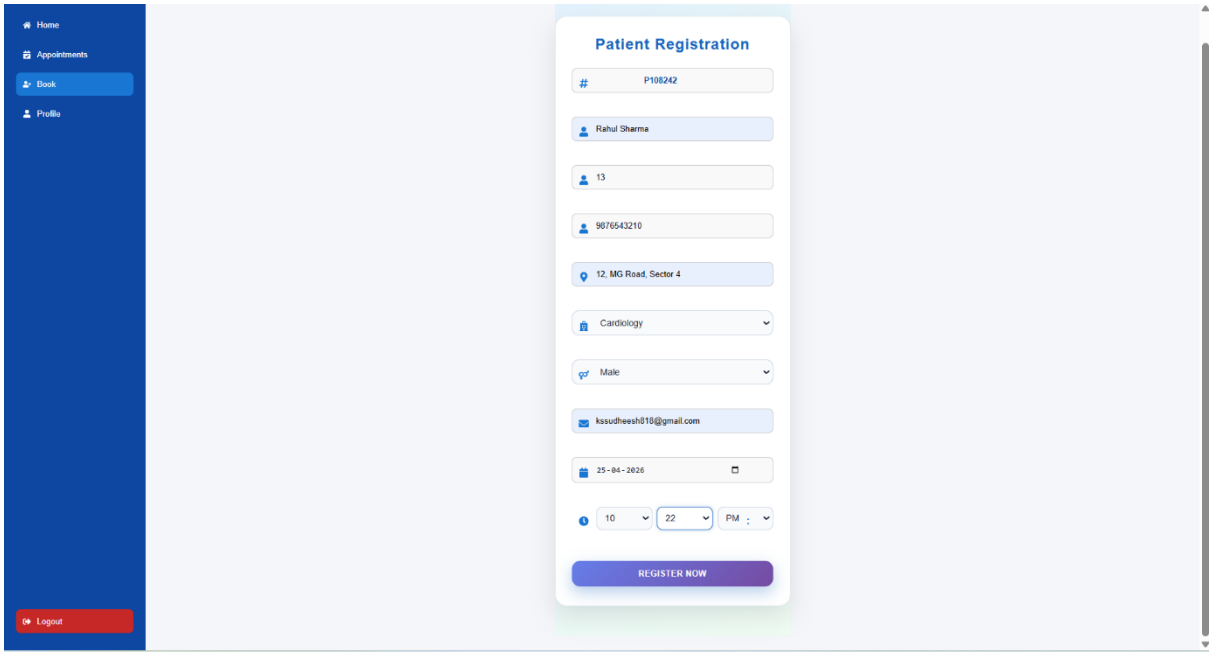
Time: 10:23 PM

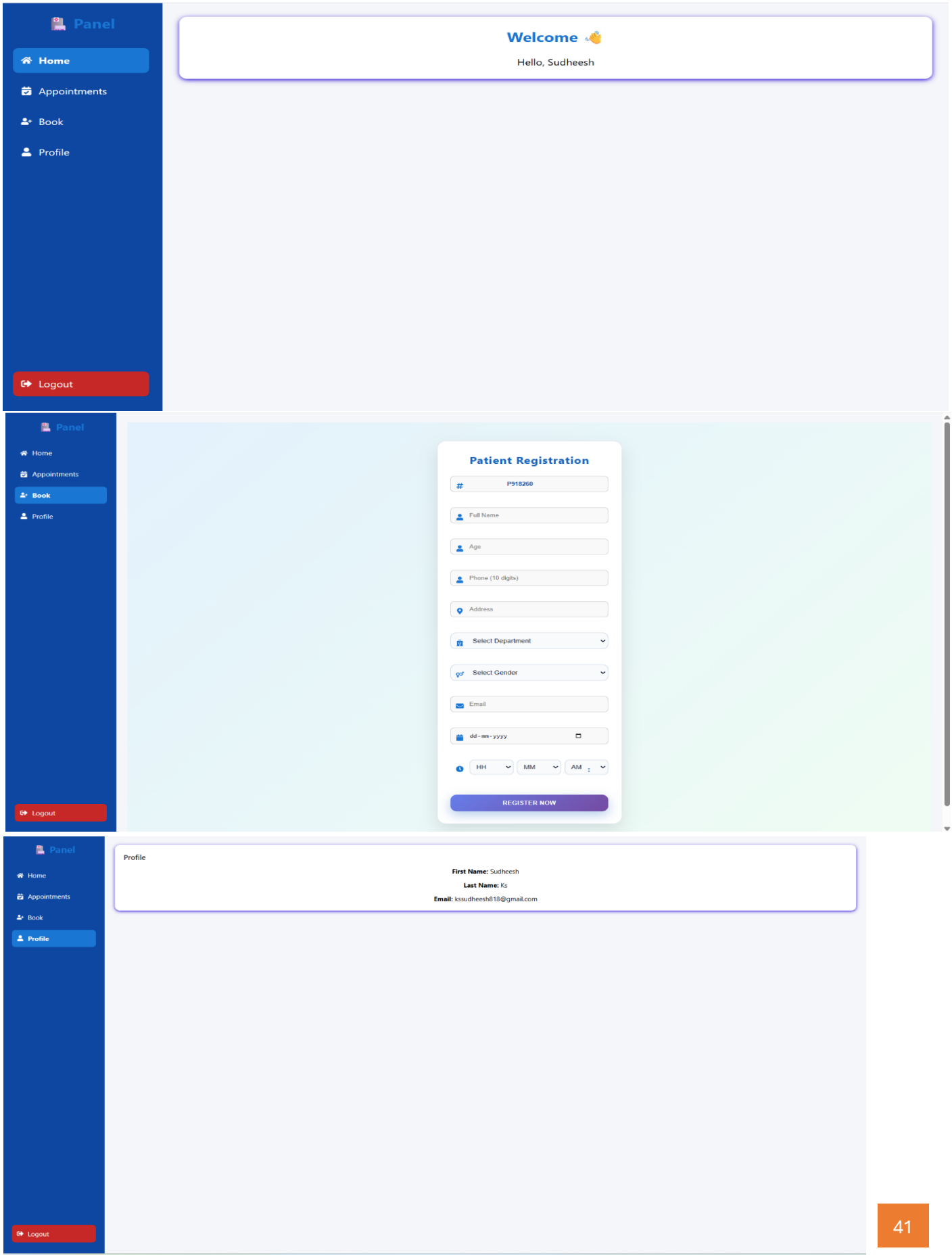
Phone: 9876543210

Address: 12, MG Road, Sector 4

DOWNLOAD PDF


Home
Appointments
Book
Profile
Logout
Search Appointment
Enter Registration Number
SEARCH
No appointment found

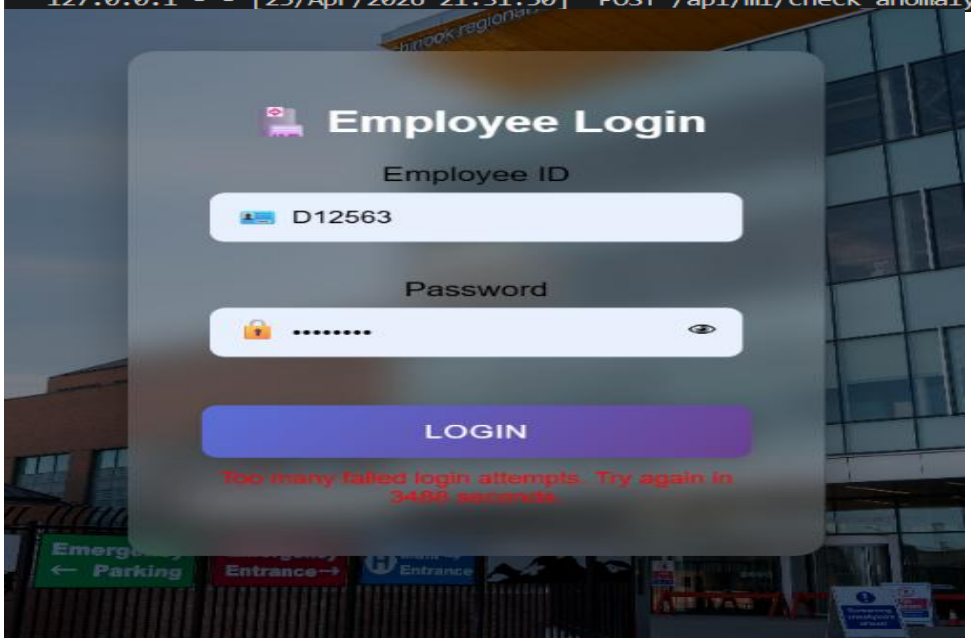

Home
Appointments
Book
Profile
Logout
Patient Registration
P108242
Rahul Sharma
13
9876543210
12, MG Road, Sector 4
Cardiology
Male
kssudheesh818@gmail.com
25-04-2026
10
22
PM
REGISTER NOW

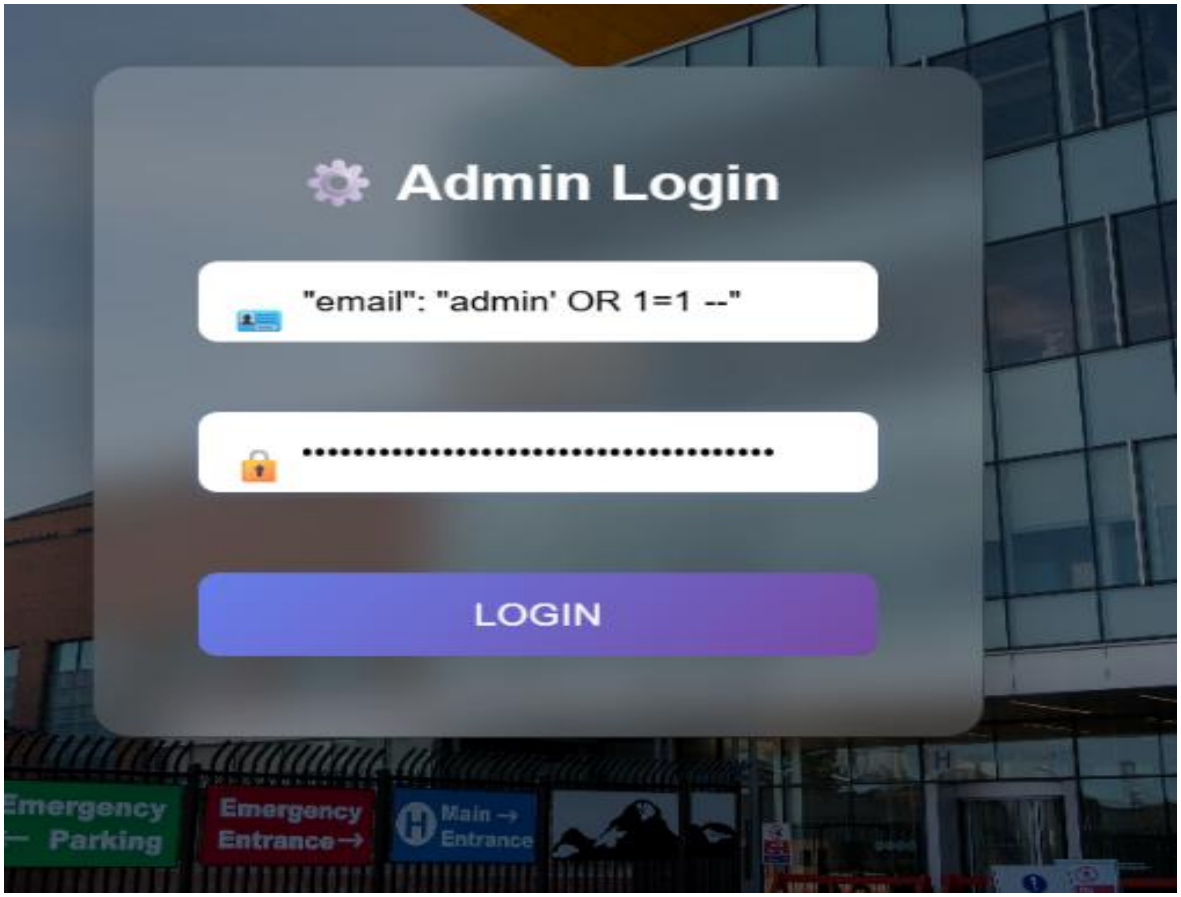

Panel
Home
Appointments
Book
Profile
Logout
Welcome
Hello, Sudheesh
Patient Registration
P918260
Full Name
Age
Phone (10 digits)
Address
Select Department
Select Gender
Email
dd-mm-yyyy
HH
MM
AM
REGISTER NOW
Profile
First Name: Sudheesh
Last Name: Ks
Email: kssudheesh818@gmail.com

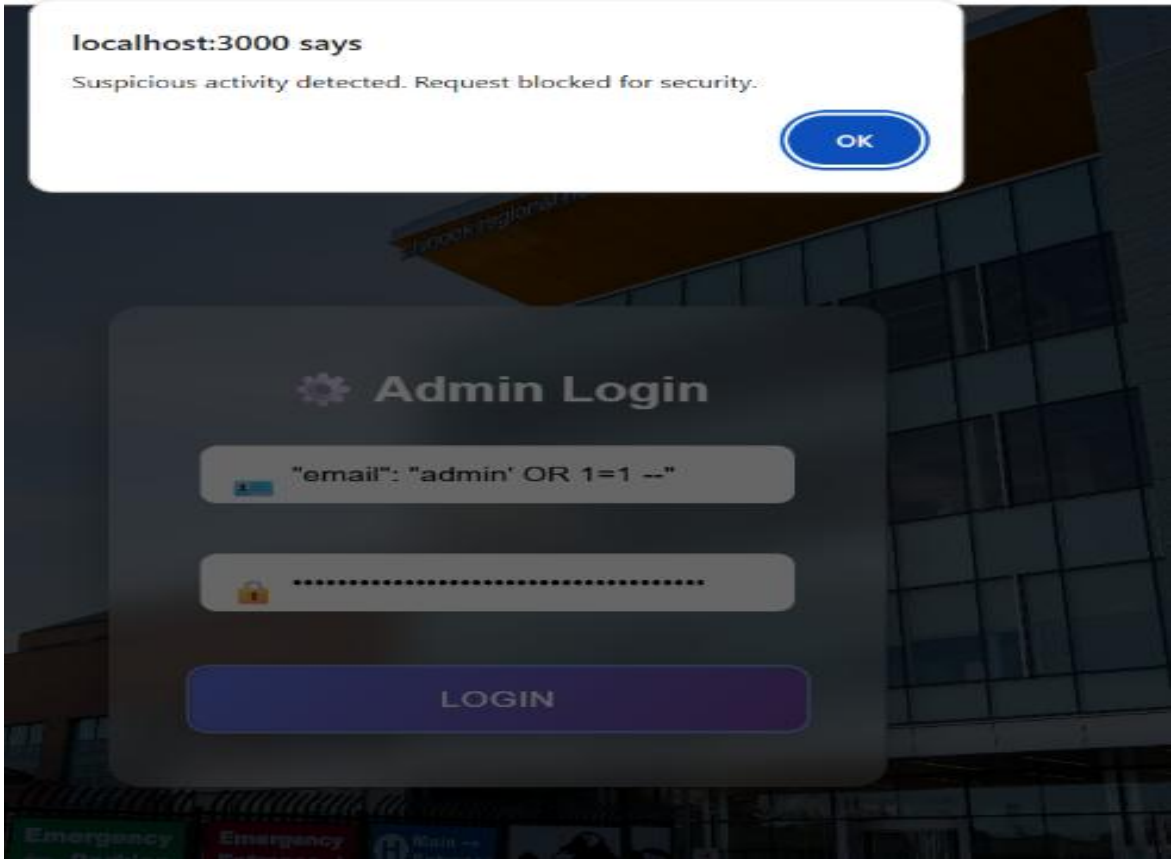
- ML Server

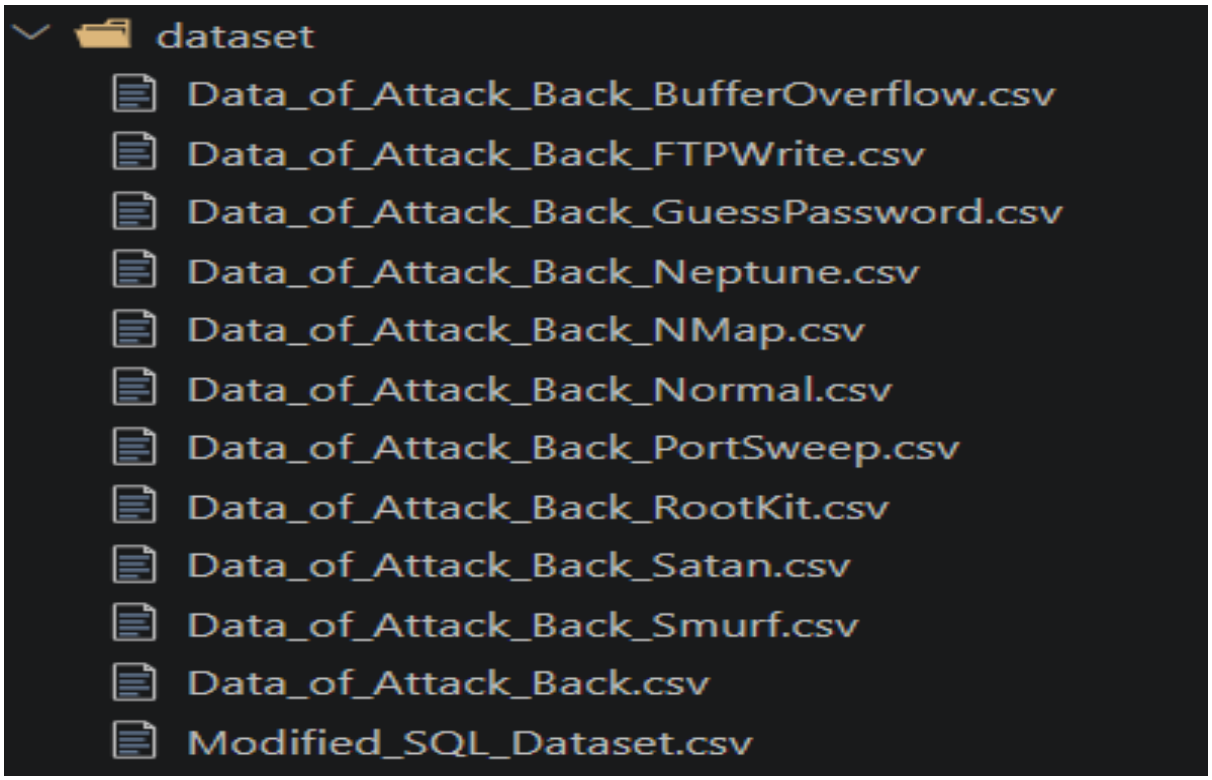
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

(.venv) PS C:\Users\USER\Desktop\project> & c:\Users\USER\Desktop\project\.venv\Scripts\
python.exe c:/Users/USER/Desktop/project/python/run_ml_api.py
📊 Dataset size: 848469
✅ Model trained successfully
🚀 Starting ML Anomaly Detection API Server...
📡 Server will be available at http://localhost:5001
🔍 Anomaly detection endpoint: POST /api/ml/check_anomaly
 * Serving Flask app 'ml_api'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://10.92.179.213:5001
Press CTRL+C to quit
127.0.0.1 - - [25/Apr/2026 13:47:34] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 13:47:45] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 13:48:49] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 13:51:17] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 14:19:10] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 18:39:58] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 19:08:28] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 19:09:01] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:01:28] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:31:17] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:31:20] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:31:29] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:31:42] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2026 21:31:50] "POST /api/ml/check_anomaly HTTP/1.1" 200 -
```




Admin Login
"email": "admin' OR 1=1 --"
LOGIN
Emergency Parking
Emergency Entrance
Main Entrance

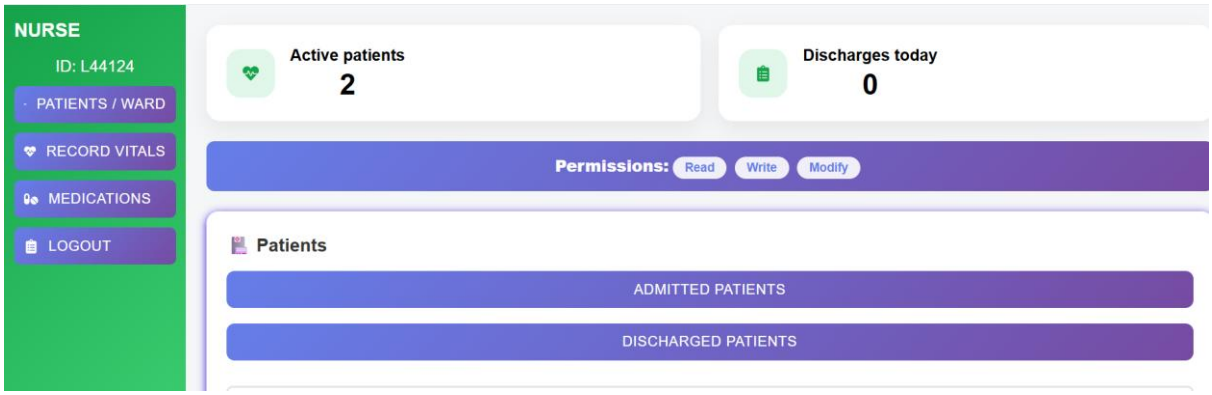

localhost:3000 says
Suspicious activity detected. Request blocked for security.
OK
Admin Login
"email": "admin' OR 1=1 --"
LOGIN

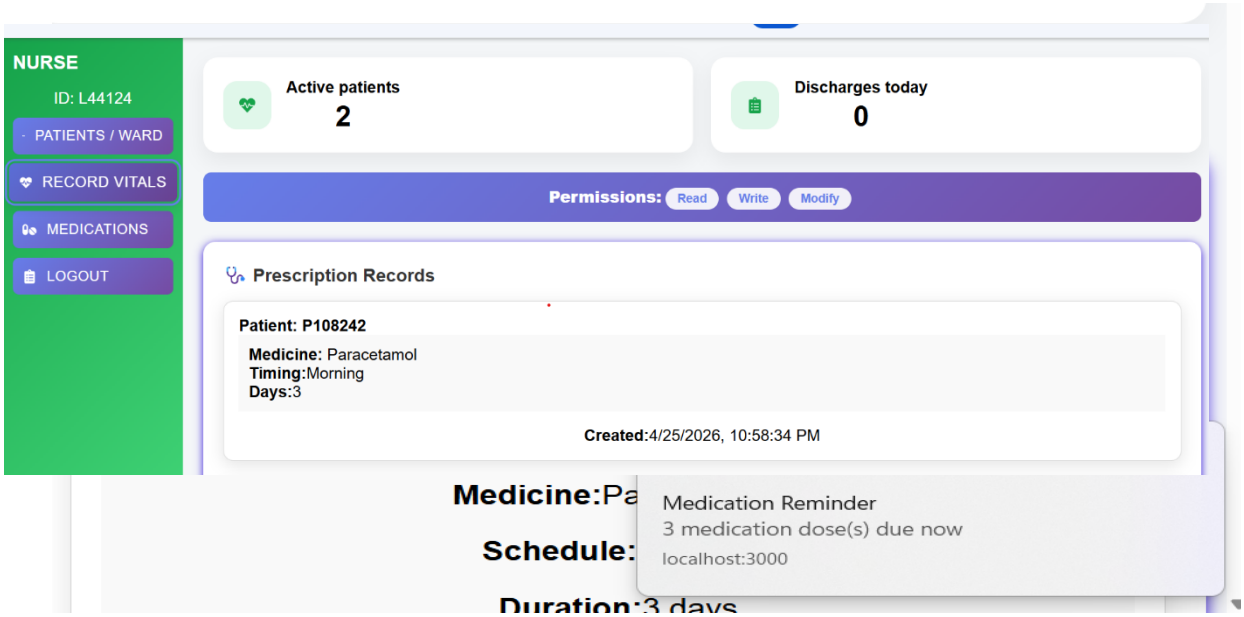


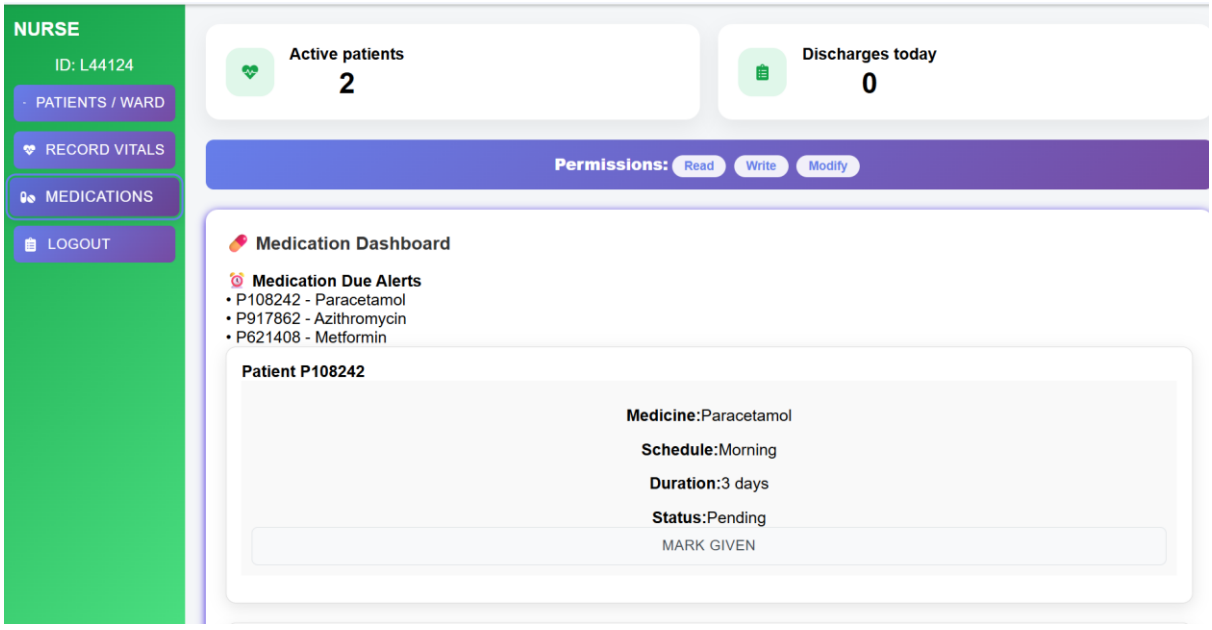
ml_api.py

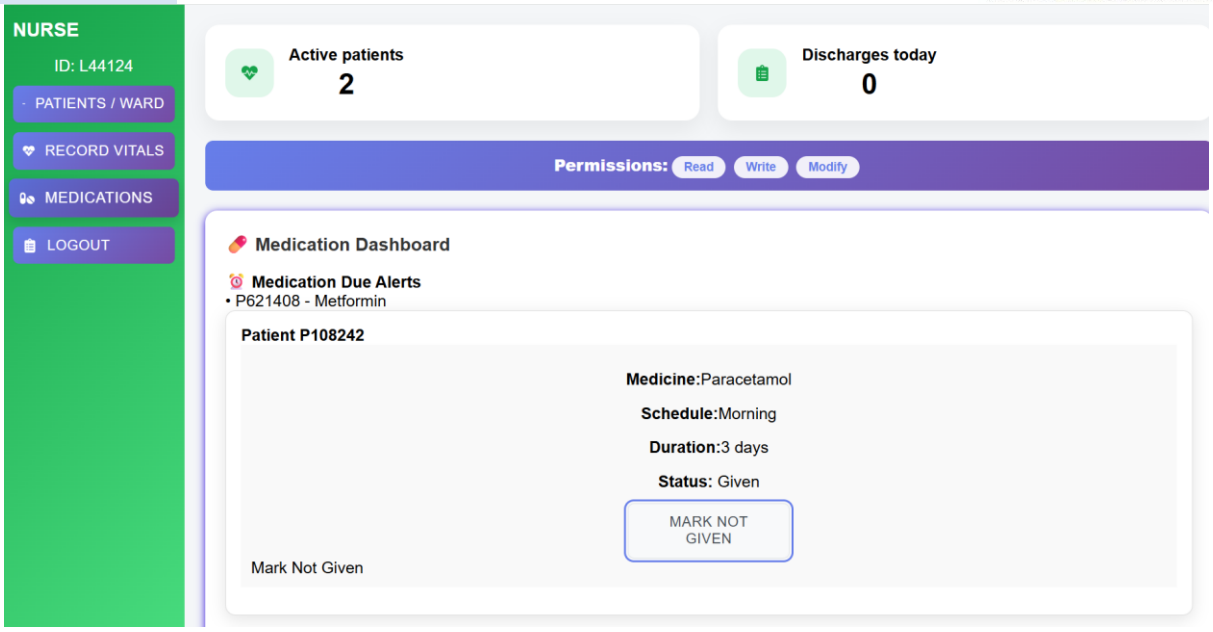
```
from flask import Flask, request, jsonify
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import joblib
import os
from datetime import datetime
import math
from collections import defaultdict
import pandas as pd
import glob
```

- NURSING


NURSE
ID: L44124
PATIENTS / WARD
RECORD VITALS
MEDICATIONS
LOGOUT
Active patients
2
Discharges today
0
Permissions: Read Write Modify
Patients
ADMITTED PATIENTS
DISCHARGED PATIENTS


NURSE
ID: L44124
PATIENTS / WARD
RECORD VITALS
MEDICATIONS
LOGOUT
Active patients
2
Discharges today
0
Permissions: Read Write Modify
Prescription Records
Patient: P108242
Medicine: Paracetamol
Timing:Morning
Days:3
Created:4/25/2026, 10:58:34 PM
Medicine:Pa
Schedule:
Duration:3 days
Medication Reminder
3 medication dose(s) due now
localhost:3000


NURSE
ID: L44124
PATIENTS / WARD
RECORD VITALS
MEDICATIONS
LOGOUT
Active patients
2
Discharges today
0
Permissions: Read Write Modify
Medication Dashboard
Medication Due Alerts
• P108242 - Paracetamol
• P917862 - Azithromycin
• P621408 - Metformin
Patient P108242
Medicine:Paracetamol
Schedule:Morning
Duration:3 days
Status:Pending
MARK GIVEN


NURSE
ID: L44124
PATIENTS / WARD
RECORD VITALS
MEDICATIONS
LOGOUT
Active patients
2
Discharges today
0
Permissions: Read Write Modify
Medication Dashboard
Medication Due Alerts
• P621408 - Metformin
Patient P108242
Medicine:Paracetamol
Schedule:Morning
Duration:3 days
Status: Given
MARK NOT GIVEN
Mark Not Given

4 Conclusions

The project successfully demonstrates the design and implementation of a secure healthcare system with an integrated identity security framework. It effectively manages users, patients, and hospital operations while ensuring strong security through authentication, authorization, and anomaly detection. The system achieves its objectives by providing a scalable, reliable, and user-friendly platform. The integration of modern technologies and security mechanisms enhances both performance and data protection, making it suitable for real-world applications.

5 Further Development or Research

The system can be enhanced in the future by integrating advanced features such as cloud deployment for better scalability and availability. Mobile application support can be added to improve accessibility for users. Advanced analytics and reporting features can be implemented to provide better insights into healthcare data. The machine learning module can be further improved with more sophisticated models for accurate threat detection. Integration with external healthcare systems and real-time notification services can also be explored to extend system capabilities.

6 References

- React.js Official Documentation
- Node.js Official Documentation
- Express.js Documentation
- PostgreSQL Documentation
- Python Flask Documentation
- JSON Web Token (JWT) Documentation
- OWASP Security Guidelines

7 Appendix

The appendix contains additional supporting materials related to the project.

- Sample API request and response formats
- Database schema details
- ER Diagram and system diagrams
- Code snippets (if required)
- Test case samples

This section provides supplementary information that supports the main report without interrupting its flow.